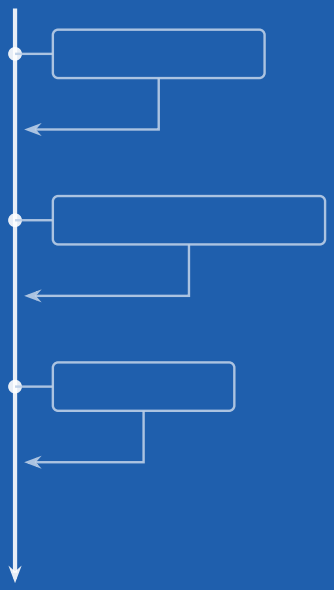
BİTİRME PROJESİ · TEKNİK RAPOR

minGPT ve FPGA Üzerinde Tamsayı Çıkarım Hızlandırıcısı

Transformer matematiğinden bit düzeyinde doğrulanmış SystemVerilog donanımına



Murat Atceken

Ekim 2026

Model	minGPT `gpt-nano` — 3 katman, 3 dikkat başı, $C = 48$, 85 728 parametre
Aritmetik	INT8 ağırlık ve aktivasyon, INT16 artık akış, yalnızca tamsayı
Doğruluk	729 / 729 girdi doğru; RTL, referans modelle 8 019 adımda bit-exact
Performans	14 544 çevrim/token — 100 MHz'de 145 µs
Model ailesi	9 mimari + stories260K, tek mikrokodlu RTL; hepsi bit-exact
Hedef kart	PYNQ-Z2 (Zynq-7020, xc7z020clg400-1)
Kaynak kod	`github.com/Muratatceken/mingpt-fpga`

Özet

Bu çalışmada Andrej Karpathy'nin minGPT modelinin `gpt-nano` yapılandırması, yalnızca tamsayı aritmetiği kullanan bir FPGA hızlandırıcısı olarak SystemVerilog ile gerçeklenmiştir. Model; 3 katman, 3 dikkat başı ve 48 boyutlu gömmeden oluşur (85 728 parametre). Model, minGPT'nin kendi örneğindeki sıralama görevinde eğitilmiştir. Kayan nokta model önce matematiksel olarak eşdeğer biçimde dönüştürülmüş, sonra INT8 ağırlık ve aktivasyonlarla, INT16 artık akışla temsil edilmiştir. LayerNorm, softmax ve GELU için yalnızca toplama, çarpma, kaydırma, taban bölme ve taban karekök kullanan tamsayı algoritmalar türetilmiş, her biri için taşma ve doyma sınırları ispatlanmıştır.

Ana sonuçlar

- **Doğruluk:** float model, tamsayı model ve RTL, görevin 729 olası girdisinin tamamında %100 doğrudur.
- **Sayısal sadakat:** tamsayı modelin logit SQNR değeri 41,5 dB'dir; float modelle token uyumu %100'dür.
- **Bit-exact doğrulama:** 729 dizi × 11 decode adımı = 8 019 adımın her birinde token, logitler ve 48 artık değer, Python referans modeliyle birebir aynıdır (Verilator 5.052).
- **Performans:** token başına 14 544 çevrim; 100 MHz'de 145 µs/token. Matris-vektör motorunun verimi %98,3'tür.
- **Bellek:** 83 KB INT8 ağırlık ve 3 KB KV önbelleği; harici bellek gerekmez.
- **Hedef kart:** PYNQ-Z2 (Zynq-7020). Çip üzeri öz-test simülasyonda doğrulanmıştır.
- **Model ailesi:** Karpathy'nin dokuz mimarisi (Bigram, MLP, RNN, GRU, BoW, GPT-2, microgpt, Llama 2, nanochat) names.txt'te eşit bütçeyle eğitilmiş ve hepsi *tek* bir mikrokodlu RTL'de bit-exact çalıştırılmıştır. INT8 kaybı her modelde 10^{-2} nat/token'ın altındadır. Token süresi 36 (Bigram) ile 10 711 (nanochat) çevrim arasındadır.
- **Hikâye üretimi:** hazır eğitilmiş stories260K (Llama 2, 260 K parametre) aynı RTL'de çalışır. RTL, tamsayı referansla token token aynı olan 200 tokenlık bir hikâye üretmiştir.

Kapsam notu. Bu raporun tarihinde Vivado sentezi, zamanlama kapanışı ve kart üzerinde çalıştırma henüz yapılmamıştır. Bunun nedeni Vivado'nun geliştirme bilgisayarı olan macOS'ta çalışmamasıdır. Gerekli bütün scriptler depoda hazırdır. Kaynak kullanımı sayıları bu yüzden açık kaynak Yosys sentezinden alınmıştır; zamanlama (Fmax) henüz ölçülmemiştir.

Rapor dört kısımdan oluşur. Kısım I (Bölüm 1–9) minGPT'nin matematiksel modelini, eğitimini ve çıkarım algoritmasını anlatır. Kısım II (Bölüm 10–14) Karpathy'nin bigram'dan nanochat'e uzanan dil modeli ailesini aynı matematiksel ve görsel dille inceler, her mimari özelliğin donanım karşılığını çıkarır ve ailenin karşılaştırma yöntemini tanımlar. Kısım III (Bölüm 15–26) minGPT'nin tamsayı aritmetiğine dönüştürülmesini, donanım mimarisini, performans analizini, doğrulamayı ve kart entegrasyonunu anlatır. Kısım IV (Bölüm 27–32) bütün aileyi çalıştıran mikrokodlu motoru, nicemleme incelemesini ve iki deney hattının sonuçlarını verir. Bütün kaynak kod ve üretilen dosyalar `github.com/Muratatceken/mingpt-fpga` adresindedir.

İçindekiler

Özet i

Kısım I minGPT: Teknik, Algoritmik ve Matematiksel Anlatım 1

1 Giriş: dil modelleme ve minGPT 1

2 Girdi temsili: token ve konum gömmeleri 2

3 Transformer bloğu I: artık akış ve LayerNorm 3

4 Transformer bloğu II: nedensel çok başlı öz-dikkat 4

5 Transformer bloğu III: MLP ve GELU 6

6 Çıkış katmanı, kayıp fonksiyonu ve eğitim 7

7 Çıkarım algoritması ve KV önbelleği 8

8 Parametre ve işlem sayısı analizi 9

9 Deney görevi: sıralama 10

Kısım II Karpathy'nin Dil Modeli Ailesi 13

10 Model ailesine genel bakış 13

11 Dikkat öncesi modeller 14

12 Transformer varyantları 16
12.1 RMSNorm . . . 16
12.2 Döner konum kodlaması (RoPE) . . . 17
12.3 Aktivasyonlar: ReLU, ReLU^2 ve SwiGLU . . . 17
12.4 Gruplanmış sorgu dikkati (GQA) . . . 18
12.5 QK-norm . . . 18
12.6 nanochat'in ek mekanizmaları ve llama2.c modeli . . . 18

13 Donanım açısından model ailesi 19

14 Karşılaştırma yöntemi 20

Kısım III FPGA İmplementasyonu 21

15 Tasarım hedefleri ve kısıtları 21

16 Sayı sistemi ve kuantizasyon teorisi 22
16.1 Düzgün simetrik kuantizasyon . . . 22
16.2 Taşıyıcı biçimler . . . 22
16.3 Yalnızca tamsayı GEMV . . . 23

16.4 Yeniden kuantizasyon (requantization) . . . 23
16.5 Kalibrasyon . . . 24
16.6 Biçim özeti ve sayısal sonuç . . . 24

17 Matematiksel olarak eşdeğer dönüşümler (katlama) 25

18 Tamsayı çekirdekler 26
18.1 Tamsayı LayerNorm . . . 26
18.2 Tamsayı softmax . . . 27
18.3 GELU: tam tablo . . . 28
18.4 Argmax ve artık toplama . . . 28

19 Donanım mimarisi 29
19.1 Üst modül arayüzü . . . 29
19.2 Kontrol FSM'i . . . 29

20 GEMV motoru 31
20.1 Dataflow seçimi . . . 31
20.2 Ağırlık ROM düzeni . . . 31
20.3 Boru hattı ve gölge banka . . . 32
20.4 Requant birimi . . . 33
20.5 Çevrim sayısı . . . 33

21 LayerNorm ve dikkat birimleri 34
21.1 Ardışık karekök . . . 34
21.2 Ardışık bölücü . . . 34
21.3 LayerNorm birimi . . . 34
21.4 Dikkat birimi . . . 35

22 Bellek organizasyonu ve FPGA kaynak eşlemesi 35

23 Performans analizi 37

24 Doğrulama metodolojisi 38

25 PYNQ-Z2 entegrasyonu 39

26 Sınırlamalar ve gelecek çalışmalar 41

Kısım IV Model Ailesi Donanımda: Mikrokodlu Motor ve Karşılaştırma 42

27 Mikrokodlu motor: tek RTL, dokuz mimari 42

28 Derleyici ve doğrulama zinciri 44

29 Nicemleme: kalibrasyon ve hata kaynakları 45

30 Hat A: names.txt üzerinde dokuz mimari 46

31 Hat B: stories260K ile hikâye üretimi 50

32 Değerlendirme 50

Kaynakça **52**

Şekil Listesi

1 minGPT ileri geçişi . . . 4
2 Nedensel maske ve çok başlı öz-dikkat . . . 5
3 KV önbellekli decode adımı . . . 9
4 Eğitilmiş modelin dikkat haritaları . . . 12
5 Karpathy'nin dil modeli depoları . . . 13
6 Dikkat öncesi dil modelleri . . . 14
7 Transformer varyantları . . . 16
8 RoPE ve GQA . . . 17
9 Çıkarım durum belleği ve bağlam . . . 19
10 Yazılımdan donanıma akış . . . 22
11 Tamsayı GEMV şeridi ve yeniden kuantizasyon . . . 23
12 Float ve tamsayı logitleri . . . 25
13 Tamsayı softmax veri yolu . . . 28
14 GELU ve INT8 GELU tablosu . . . 28
15 `gpt_top` veri yolu . . . 29
16 Kontrol FSM'i . . . 30
17 GEMV motoru . . . 32
18 GEMV boru hattı zamanlaması . . . 33
19 Token başına çevrim profili . . . 37
20 Paralellik derecesinin etkisi . . . 38
21 Referans model hiyerarşisi . . . 38
22 PYNQ-Z2 üst modülü . . . 40
23 Mikrokodlu zoo motoru . . . 43
24 Eğitim eğrileri . . . 46
25 Model başına çevrim profili . . . 48
26 Kalite ve hız . . . 49

KISIM I

minGPT: Teknik, Algoritmik ve Matematiksel Anlatım

1 Giriş: dil modelleme ve minGPT

Bir dil modeli, sonlu bir sözlük $\mathcal{V}$ üzerindeki token dizilerine olasılık atayan bir fonksiyondur. Olasılığın zincir kuralı, bu ortak dağılımı koşullu dağılımların çarpımına kesin olarak ayırır:

$$p(x_1, x_2, \dots, x_T) \;=\; \prod_{t=1}^{T} p\left(x_t \mid x_1, \dots, x_{t-1}\right). \tag{1}$$

Bu ayrışım bir yaklaşım değil, olasılık teorisinin bir özdeşliğidir. Otoregresif bir model her koşullu dağılımı parametreleri θ olan tek bir sinir ağı f_θ ile modeller:

$$p_\theta\left(x_t \mid x_{<t}\right) = \operatorname{softmax}\left(f_\theta(x_{<t})\right)_{x_t}, \qquad f_\theta : \mathcal{V}^{t-1} \to \mathbb{R}^{|\mathcal{V}|}. \tag{2}$$

f_θ'nın çıktısı olan $|\mathcal{V}|$ boyutlu vektöre *logit* vektörü denir. Metin üretmek için bu dağılımdan bir token seçilir, diziye eklenir ve işlem tekrarlanır.

GPT ailesi. GPT (*Generative Pre-trained Transformer*), f_θ olarak Vaswani ve ark. [1] tarafından önerilen transformer mimarisinin yalnızca kod çözücü (*decoder-only*) yarısını kullanır. GPT-1 [2], GPT-2 [3] ve GPT-3 [4] aynı temel yapıyı korur. Aralarındaki fark esas olarak katman sayısı, gömme boyutu ve eğitim verisidir. Kod çözücü bloklarda kodlayıcıya yönelik çapraz dikkat bulunmaz; yalnızca nedensel öz-dikkat ve ileri beslemeli ağ vardır.

minGPT. minGPT [5], GPT-2 ile aynı mimariye sahip, yaklaşık 300 satırlık bir PyTorch gerçeklemesidir. Depo üç temel dosyadan oluşur:

- `mingpt/model.py`: model tanımı (`GPT`, `Block`, `CausalSelfAttention`, `NewGELU`) ve `generate()` fonksiyonu,

- `mingpt/trainer.py`: genel amaçlı eğitim döngüsü,
- `mingpt/bpe.py`: GPT-2'nin bayt düzeyinde BPE tokenizer'ı.

minGPT eğitim amaçlıdır, ancak mimari olarak eksiksizdir: `GPT.from_pretrained()` ile OpenAI'ın GPT-2 ağırlıklarını yükleyip aynı çıktıları üretebilir. Bu nedenle minGPT üzerinde yapılan bir donanım tasarımı, büyük GPT modellerinin hesap yapısını da temsil eder. Bir token için yapılan işlemler (matris-vektör çarpımı, LayerNorm, softmax, GELU) GPT-2'de de aynıdır; değişen yalnızca boyutlardır.

Bu çalışmanın kapsamı. Model GPU/CPU üzerinde float32 ile eğitilir. FPGA üzerinde yalnızca *çıkarım* yapılır: verilen bir önek dizisinden sonraki tokenların üretilmesi. Bu ayrım güncel uygulamalardaki iş bölümüyle uyumludur. Eğitim bir kez ve yüksek hassasiyetle yapılır; çıkarım ise milyonlarca kez, düşük enerji ve gecikmeyle yapılmalıdır.

Rapor boyunca şu notasyon kullanılır: C gömme boyutu (`n_embd` $= 48$), H dikkat başı sayısı (3), $d_h = C/H$ baş boyutu (16), L katman sayısı (3), V sözlük boyutu (3) ve T en uzun bağlam (`block_size` $= 11$).

2 Girdi temsili: token ve konum gömmeleri

Transformer ham metin değil, tamsayı token kimlikleri alır. Her token, iki tablodan okunan vektörlerin toplamına dönüştürülür.

Tokenizasyon. GPT-2 bayt düzeyinde BPE (*Byte Pair Encoding*) kullanır ve sözlüğü 50 257 tokendan oluşur. Bu projedeki sıralama görevinde sözlük yalnızca $\{0, 1, 2\}$ rakamlarıdır ($V = 3$). Her rakam doğrudan kendi token kimliğidir, bu yüzden ayrı bir tokenizer gerekmez.

Token gömmesi. Öğrenilebilir bir matris $W_{te} \in \mathbb{R}^{V \times C}$ her token kimliğini C boyutlu bir vektöre eşler. Bu işlem, one-hot kodlanmış bir vektörle matris çarpımına eşdeğer olan bir tablo okumasıdır:

$$e(\tau) = W_{te}[\tau, :] = W_{te}^{\top} \mathbf{1}_{\tau}, \qquad \mathbf{1}_{\tau} \in \{0, 1\}^{V}. \tag{3}$$

Donanım açısından bu eşdeğerlik önemlidir: gömme katmanı hiç çarpıcı gerektirmez, yalnızca bir ROM okumasıdır.

Konum gömmesi. Maskesiz öz-dikkat, konum bilgisi verilmezse girdinin sırasına duyarsızdır. Bir permütasyon matrisi P için $\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X)$ olur; dikkat *permütasyona eşdeğişkendir*. Bu durumda model "0 1 2" ile "2 1 0" dizilerini ayırt edemez. Nedensel maske sıra hakkında yalnızca dolaylı ve kısmi bir bilgi verir; bu yüzden açık bir konum kodlaması kullanılır. minGPT, GPT-2 gibi, öğrenilebilir bir konum tablosu $W_{pe} \in \mathbb{R}^{T \times C}$ kullanır. Bu tablo, orijinal transformer'daki sabit sinüzoidal kodlamanın yerine geçer.

Girdi matrisi. Uzunluğu T olan bir dizi için birinci bloğa giren matris şudur:

$$X^{(0)} = \begin{bmatrix} W_{te}[\tau_0] + W_{pe}[0] \\ W_{te}[\tau_1] + W_{pe}[1] \\ \vdots \\ W_{te}[\tau_{T-1}] + W_{pe}[T-1] \end{bmatrix} \in \mathbb{R}^{T \times C}. \tag{4}$$

Eğitim sırasında bu toplama `embd_pdrop` $= 0{,}1$ oranında dropout uygulanır; çıkarımda dropout devre dışıdır. Bu projede W_{te} 3×48, W_{pe} 11×48 boyutundadır ve iki tablo toplam 672 parametre içerir.

3 Transformer bloğu I: artık akış ve LayerNorm

minGPT'nin gövdesi L adet özdeş transformer bloğunun art arda bağlanmasından oluşur. Her blok iki alt katman içerir ve her alt katmanın çıktısı girdisine eklenir (*residual connection*). minGPT, GPT-2 gibi **pre-LN** yapıyı kullanır; LayerNorm alt katmanın girişinde uygulanır:

$$X' = X + \mathrm{Attn}\left(\mathrm{LN}_1(X)\right), \qquad X'' = X' + \mathrm{MLP}\left(\mathrm{LN}_2(X')\right). \tag{5}$$

`model.py`'de bu iki satır `Block.forward()` içinde birebir `x = x + self.attn(self.ln_1(x))` ve `x = x + self.mlpf(self.ln_2(x))` olarak yazılıdır. Son bloğun ardından bir son LayerNorm (`ln_f`) daha uygulanır. Şekil 1 bütün ileri geçişi gösterir.

Artık akış. X matrisi bloklar boyunca taşınan bir "ana veri yolu" gibi düşünülebilir. Her alt katman bu yoldan okur, bir düzeltme hesaplar ve sonucu yola geri ekler. L blok sonunda akış $2L$ düzeltmenin toplamını taşır:

$$X^{(L)} = X^{(0)} + \sum_{\ell=1}^{L} \left[\Delta_\ell^{\mathrm{attn}} + \Delta_\ell^{\mathrm{mlp}} \right]. \tag{6}$$

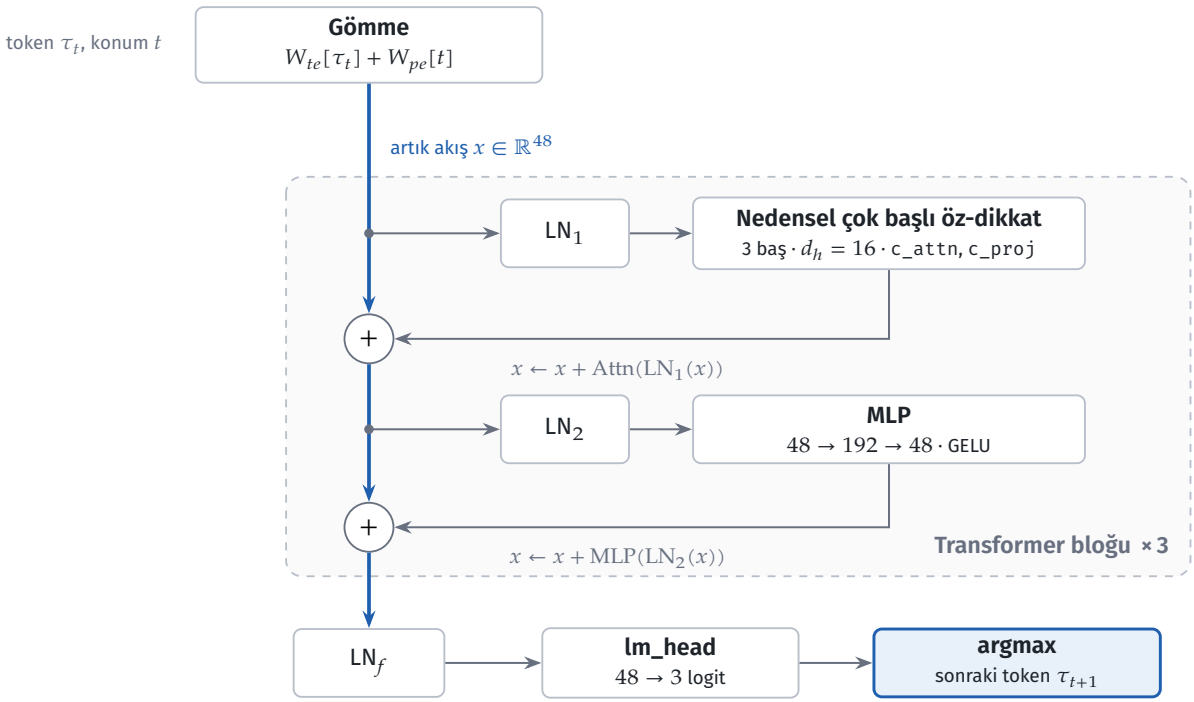
Bu yapının iki sonucu vardır. Birincisi, $\partial X'' / \partial X = I + (\text{alt katman türevleri})$ olduğu için birim matris terimi gradyanın derin ağlarda sönmeden geri yayılmasını sağlar [6]. İkincisi donanımla ilgilidir: akış, ağın en geniş dinamik aralığa sahip tensörüdür, çünkü düzeltmeler birikir. Kısım III'te bu yüzden artık akış INT8 yerine INT16 ile temsil edilecektir.

Pre-LN ve post-LN. Orijinal transformer'da LayerNorm toplamadan sonra uygulanır (post-LN). Xiong ve ark. [8], pre-LN yapıda öğrenme oranı ısınması (*warm-up*) olmadan da kararlı eğitim yapılabildiğini göstermiştir. Pre-LN'de artık yol üzerinde hiçbir normalizasyon bulunmaz.

LayerNorm. LayerNorm [7] her token vektörünü kendi C özelliği üzerinden normalize eder ve batch boyutundan bağımsızdır:

$$\mu = \frac{1}{C}\sum_{i=1}^{C} x_i, \qquad \sigma^2 = \frac{1}{C}\sum_{i=1}^{C} (x_i - \mu)^2, \qquad \mathrm{LN}(x)_i = \gamma_i \, \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i. \tag{7}$$

Burada $\gamma, \beta \in \mathbb{R}^C$ öğrenilebilir parametrelerdir ve $\epsilon = 10^{-5}$'tir. Normalize edilmiş ara vektörü $\hat{z} = (x - \mu)/\sqrt{\sigma^2 + \epsilon}$ ile gösterelim. Kısım III'te kullanılacak üç özellik şunlardır:



Şekil 1. minGPT ileri geçişi (`gpt-nano`). Artık akış (mavi) blokların içinden kesintisiz geçer. LayerNorm yalnızca alt katmana giden kolda yer alır (pre-LN). Üç blok art arda uygulanır; ardından LN_f, `lm_head` ve argmax gelir.

- **Ölçek ve kaydırma değişmezliği:** $a > 0$ ve b sabiti için $\hat{z}(ax+b) \approx \hat{z}(x)$ olur (ϵ ihmal edilince eşitlik tamdır). Girdinin sabit nokta ölçeği bu yüzden normalizasyon sonucunu etkilemez.
- **Küre üzerinde olma:** $\sum_i \hat{z}_i = 0$ ve $\sum_i \hat{z}_i^2 \approx C$; yani $\hat{z}$, yarıçapı $\sqrt{C}$ olan bir kürenin üzerindedir.
- **Sınırlılık:** bu iki koşul her bileşen için $|\hat{z}_i| \le \sqrt{C-1}$ sonucunu verir (Önerme 6). Bu sınır veriden bağımsızdır.

Afin kısım (γ, β) normalizasyondan sonra geldiği için, Bölüm 17'de bir sonraki doğrusal katmana katlanarak donanımdan tamamen kaldırılacaktır.

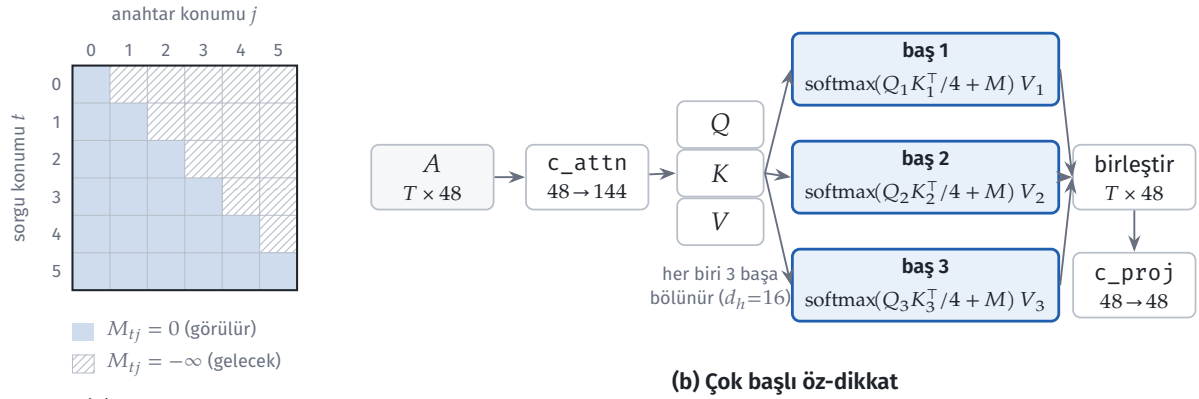
4 Transformer bloğu II: nedensel çok başlı öz-dikkat

Öz-dikkat, transformer'da tokenların birbirinden bilgi aldığı tek mekanizmadır. Her token, önceki tokenların değer vektörlerinin ağırlıklı ortalamasını alır. Ağırlıklar, sorgu ile anahtarların benzerliğinden hesaplanır.

Sorgu, anahtar, değer. LN_1 çıktısı $A \in \mathbb{R}^{T\times C}$, tek bir doğrusal katmanla (`c_attn`, $C \to 3C$) üç matrise dönüştürülür:

$$[\,Q \mid K \mid V\,] = A\,W_{attn}^{\top} + b_{attn}, \qquad W_{attn} \in \mathbb{R}^{3C\times C}, \quad Q,K,V \in \mathbb{R}^{T\times C}. \tag{8}$$

PyTorch'un `nn.Linear` katmanı $y = xW^{\top} + b$ hesaplar; ağırlık (çıkış × giriş) biçiminde saklanır. Donanımdaki ağırlık ROM'u da bu satır düzenini kullanır.



(a) Nedensel maske **(b) Çok başlı öz-dikkat**

Şekil 2. (a) $T = 6$ için nedensel maske: t konumu yalnızca $j \le t$ konumlarını görür. (b) Çok başlı öz-dikkat: `c_attn` çıktısı üç başa bölünür; başların çıktıları birleştirilip `c_proj`'dan geçirilir. $1/\sqrt{16} = 1/4$ çarpanı Bölüm 17'de ağırlıklara katlanır.

Başlara bölme. C boyutu, her biri $d_h = C/H$ boyutlu H başa bölünür. h numaralı baş Q, K, V'nin $[h\,d_h, (h+1)\,d_h)$ aralığındaki sütunlarını kullanır. Her baş, ölçekli iç çarpım dikkatini bağımsız olarak hesaplar:

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_h}} + M\right) V_h, \qquad M_{tj} = \begin{cases} 0, & j \le t, \\ -\infty, & j > t. \end{cases} \tag{9}$$

Softmax satır bazlıdır: $\text{softmax}(s)_j = e^{s_j} / \sum_k e^{s_k}$. Sonuç negatif olmayan ve toplamı 1 olan bir olasılık vektörüdür. Şekil 2 maskeyi ve çok başlı yapıyı gösterir.

Neden $\sqrt{d_h}$ ile bölünür? Sorgu ve anahtar bileşenlerinin bağımsız, sıfır ortalamalı ve birim varyanslı olduğunu varsayalım. Bu durumda iç çarpımın varyansı boyutla doğrusal büyür:

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_h} \text{Var}(q_i k_i) = \sum_{i=1}^{d_h} \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] = d_h. \tag{10}$$

$\sqrt{d_h}$'ye bölmek skorları yeniden birim varyansa getirir. Bölme yapılmasa büyük d_h'de skorlar çok büyür, softmax tek bir girdiye doyar ve gradyan sıfıra yaklaşır [1].

Nedensel maske. M matrisi, t konumunun yalnızca $j \le t$ konumlarına bakmasını sağlar. $e^{-\infty} = 0$ olduğu için gelecekteki tokenların ağırlığı tam olarak sıfırdır. Bu sayede eğitimde T konumun hepsi paralel hesaplanır, ama her konum yalnızca geçmişi görür.

Softmax'ın iki özelliği. Donanım tasarımı için kritik olan iki özellik vardır:

1. **Kaydırma değişmezliği:** her c sabiti için $\text{softmax}(s - c) = \text{softmax}(s)$. $c = \max_j s_j$ seçilirse bütün üsler sıfır veya negatif olur ve $e^x \in (0, 1]$ aralığında kalır; taşma imkânsızdır.
2. **Dışbükey birleşim:** $p_j \ge 0$ ve $\sum p_j = 1$ olduğundan başın çıktısı $y = \sum_j p_j v_j$, değer vektörlerinin dışbükey birleşimidir: her bileşen için $\min_j v_{j,d} \le y_d \le \max_j v_{j,d}$.

Birleştirme ve çıkış izdüşümü. Başların çıktıları yan yana konur ve ikinci bir doğrusal katmandan (`c_proj`, $C \to C$) geçirilir:

$$\mathrm{Attn}(A) = [\,\mathrm{head}_1 \mid \cdots \mid \mathrm{head}_H\,]\, W_{proj}^{\top} + b_{proj}. \tag{11}$$

Tensör boyutları. B batch boyutu olmak üzere `CausalSelfAttention.forward()` Tablo 1'daki dönüşümleri yapar. Tam bir dizi için skor matrisi T^2 elemanlıdır; dikkatin maliyeti katman başına $O(T^2C)$'dir. Bölüm 7'de çıkarımda bu maliyetin token başına $O(tC)$'ye nasıl indiği gösterilecektir.

Tablo 1. `CausalSelfAttention.forward()` içindeki tensör dönüşümleri.

Adım	İşlem	Boyut
1	`c_attn(x).split(C)`	$3 \times (B,T,C)$
2	`view(B,T,H,d_h).transpose(1,2)`	(B,H,T,d_h)
3	`q @ k.transpose(-2,-1) / sqrt(d_h)`	(B,H,T,T)
4	maske + softmax + dropout	(B,H,T,T)
5	`att @ v`	(B,H,T,d_h)
6	`transpose(1,2).view(B,T,C)`	(B,T,C)
7	`c_proj` + dropout	(B,T,C)

5 Transformer bloğu III: MLP ve GELU

Bloğun ikinci alt katmanı, her tokene ayrı ayrı uygulanan iki katmanlı bir ileri beslemeli ağdır:

$$\mathrm{MLP}(a) = W_2\,\mathrm{GELU}\,(W_1 a + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{4C\times C}, \quad W_2 \in \mathbb{R}^{C\times 4C}. \tag{12}$$

minGPT'de W_1 `mlp.c_fc`, W_2 ise `mlp.c_proj` katmanıdır. Ara genişlik $4C = 192$'dir. MLP, bloğun $12C^2$ ağırlığının $8C^2$'sini, yani üçte ikisini taşır; hesap yükünün büyük kısmı da buradadır.

GELU. Hendrycks ve Gimpel'in [9] önerdiği GELU, girdiyi standart normal dağılımın birikimli dağılım fonksiyonu Φ ile ağırlıklandırır:

$$\mathrm{GELU}(x) = x\,\Phi(x) = \frac{x}{2}\left(1 + \mathrm{erf}\!\left(\frac{x}{\sqrt{2}}\right)\right). \tag{13}$$

ReLU girdiyi işaretine göre sert biçimde keser. GELU ise bu kesmeyi yumuşak ve türevlenebilir yapar. minGPT, GPT-2 ile aynı sonuçları vermek için `NewGELU` sınıfında tanh yaklaşımını kullanır:

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left(1 + \tanh\!\left(\sqrt{2/\pi}\,\left(x + 0{,}044715\,x^3\right)\right)\right). \tag{14}$$

Bu projede sayısal olarak hesaplanan değerler şunlardır: tanh yaklaşımı ile erf tanımı arasındaki en büyük mutlak fark $4{,}73\times10^{-4}$'tür ($x \approx 2{,}70$). GELU monoton değildir; en küçük değeri yaklaşık $-0{,}170$'tir ve $x \approx -0{,}752$'de alınır.

Donanım açısından erf ve tanh, doğrudan gerçeklenmesi pahalı aşkın fonksiyonlardır. Bölüm 18'te girişin INT8'e kuantize edilmesi sayesinde GELU'nun 256 girişlik bir tabloyla, yaklaşım hatası olmadan gerçeklendiği gösterilecektir (Şekil 14).

Dropout. Eğitimde MLP çıktısına `resid_pdrop` $= 0{,}1$ oranında dropout uygulanır. Dropout her elemanı p olasılığıyla sıfırlar, kalanları $1/(1-p)$ ile ölçekler; böylece beklenen değer çıkarımdakiyle aynı kalır. Çıkarımda (`model.eval()`) dropout birim dönüşümdür ve donanımda karşılığı yoktur.

6 Çıkış katmanı, kayıp fonksiyonu ve eğitim

Son bloğun çıktısı `ln_f` ile normalize edilir ve bias içermeyen `lm_head` katmanıyla sözlük boyutuna izdüşürülür. minGPT, GPT-2'nin aksine `lm_head` ağırlığını token gömmesiyle paylaşmaz:

$$z_t = W_{head}\,\mathrm{LN}_f\left(x_t^{(L)}\right) \in \mathbb{R}^V, \qquad p_\theta(\cdot \mid x_{\le t}) = \mathrm{softmax}(z_t). \tag{15}$$

Kayıp fonksiyonu. Eğitim, hedef tokenların negatif log-olabilirliğini (çapraz entropi) en aza indirir. Hedefi -1 olan konumlar kayba katılmaz (`ignore_index=-1`). İşaretli konumların kümesi $\mathcal{M}$ olmak üzere:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{M}|}\sum_{(b,t)\in\mathcal{M}} \log p_\theta\left(y_{b,t} \mid x_{b,\le t}\right). \tag{16}$$

Çapraz entropinin logitlere göre gradyanı basittir: $\partial\mathcal{L}/\partial z_t = p_t - e_{y_t}$. Rastgele tahmin yapan bir modelin kaybı $\ln V$'dir; $V = 3$ için 1,0986.

Öğretmen zorlaması. Eğitimde girdi dizisi x ve bir konum kaydırılmış hedef dizisi y birlikte verilir. Nedensel maske sayesinde T konumun kayıp terimleri tek bir ileri geçişte paralel hesaplanır.

Optimizasyon: AdamW. minGPT, Loshchilov ve Hutter'ın [10] ayrık ağırlık azaltmalı Adam yöntemini kullanır. g_t gradyan, η öğrenme oranı ve λ ağırlık azaltma katsayısı olmak üzere:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\,g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2)\,g_t^2, \\ \hat{m}_t &= m_t/(1-\beta_1^t), & \hat{v}_t &= v_t/(1-\beta_2^t), \\ \theta_t &= \theta_{t-1} - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} + \lambda\,\theta_{t-1}\right). \end{aligned} \tag{17}$$

Tablo 2. minGPT'nin optimizasyon ayarları (`trainer.py`, `model.py::configure_optimizers`).

Ayar	Değer
β_1, β_2	0,9 ve 0,95
Ağırlık azaltma λ	0,1, yalnızca `nn.Linear` ağırlıklarına
Azaltma uygulanmayanlar	tüm bias'lar, LayerNorm ve gömme ağırlıkları
Gradyan kırpma	norm $\le 1{,}0$ (`clip_grad_norm_`)
Batch boyutu	64
Öğrenme oranı	varsayılan 3×10^{-4}; bu projede 5×10^{-4}

Ağırlık başlatma. Tüm doğrusal ve gömme ağırlıkları $\mathcal{N}(0, 0{,}02^2)$ dağılımından çekilir. Bias'lar 0, LayerNorm'un γ'sı 1 ve β'sı 0 ile başlar. Artık akışa yazan izdüşümlerin (`c_proj`) standart sapması ise $0{,}02/\sqrt{2L}$ alınır. Akış $2L$ düzeltmenin toplamıdır ve bağımsız terimlerin varyansları toplanır; bu ölçekleme akışın varyansını derinlikten bağımsız tutar [3].

7 Çıkarım algoritması ve KV önbelleği

Çıkarımda model, verilen bir önek diziyi her adımda bir token uzatır. minGPT'nin `generate()` fonksiyonu bunu her adımda tüm diziyi yeniden işleyerek yapar. Bu çalışmadaki tamsayı model ve donanım ise yalnızca yeni tokenı işler; geçmiş KV önbelleğinde tutulur.

`generate()` algoritması. Her adımda şu işlemler yapılır:

1. Bağlam `block_size` uzunluğunu aşarsa son `block_size` token alınır.
2. Bağlamın tamamı için ileri geçiş yapılır ve yalnızca son konumun logitleri z alınır.
3. z sıcaklığa bölünür; istenirse yalnızca en büyük k logit tutulur (*top-k*).
4. $p = \mathrm{softmax}(z)$ hesaplanır; sonraki token örneklenir veya açgözlü olarak $\arg\max_j p_j$ seçilir.
5. Seçilen token diziye eklenir.

Açgözlü kod çözmede softmax gereksizdir. Softmax kesin artan bir fonksiyon olduğundan $\arg\max \mathrm{softmax}(z) = \arg\max z$ sağlanır; donanım bu özelliği kullanır ve çıkışta softmax hesaplamaz.

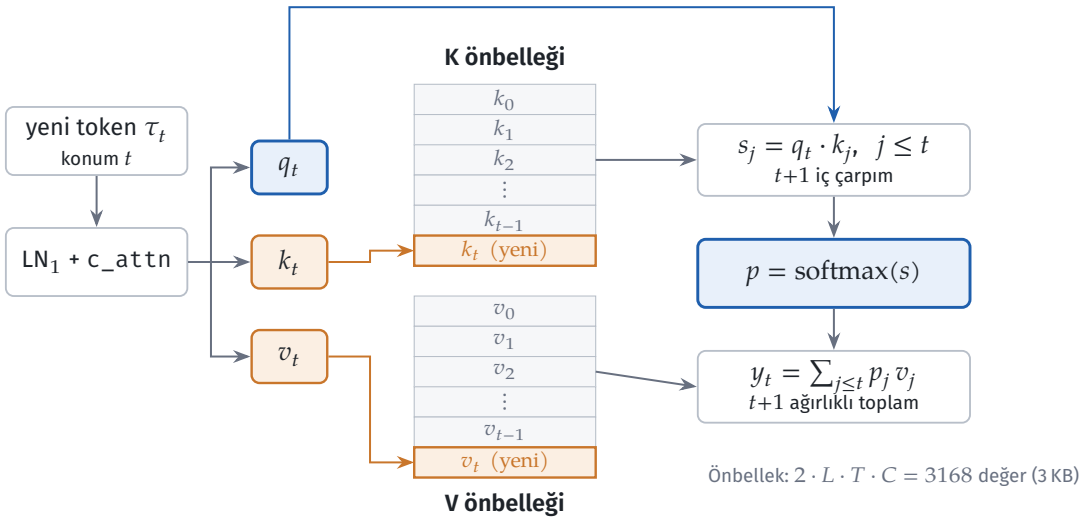
> **Önerme 1** (Nedensel bağımlılık)**.** Nedensel maske altında j konumundaki her ara değer (her katmandaki x_j, k_j, v_j) yalnızca $\tau_0, \dots, \tau_j$ tokenlarına bağlıdır.

İspat. Katman sırası üzerinden tümevarım. Gömmede $x_j^{(0)}$ yalnızca τ_j ve j'ye bağlıdır. Bir bloğun girişinde iddianın doğru olduğunu varsayalım. LayerNorm, doğrusal katmanlar, GELU ve artık toplama konum bazlıdır; j konumunun çıktısını yalnızca j konumunun girdisinden üretir. Dikkat ise j konumunda yalnızca $i \le j$ konumlarının k_i, v_i değerlerini kullanır ve bunlar varsayım gereği yalnızca $\tau_0, \dots, \tau_i$'ye bağlıdır. Dolayısıyla iddia bloğun çıkışında da doğrudur. □

Sonuç: KV önbelleği. Diziye yeni bir token eklendiğinde önceki konumların k_j, v_j vektörleri değişmez. Bunlar bir kez hesaplanıp saklanırsa, t adımında yalnızca t konumu işlenir (Şekil 3):

$$
\begin{aligned}
[\, q_t \,|\, k_t \,|\, v_t \,] &= W_{attn}\, \mathrm{LN}_1(x_t) + b_{attn} \qquad (k_t, v_t \text{ önbelleğe yazılır}), \\
y_t^{(h)} &= \sum_{j=0}^{t} \frac{\exp\!\left(q_t^{(h)} \cdot k_j^{(h)} / \sqrt{d_h}\right)}{\sum_{i=0}^{t} \exp\!\left(q_t^{(h)} \cdot k_i^{(h)} / \sqrt{d_h}\right)}\, v_j^{(h)}.
\end{aligned}
\tag{18}
$$

Bu formda bütün doğrusal katmanlar matris-matris çarpımı (GEMM) yerine **matris-vektör çarpımına (GEMV)** dönüşür. Dikkat de $t+1$ uzunluğunda iki iç çarpım dizisine indirgenir.



Şekil 3. KV önbellekli bir decode adımı (bir katman, bir baş). Yeni token yalnızca kendi q_t, k_t, v_t vektörlerini üretir; k_t ve v_t önbelleğin t satırına yazılır. Dikkat, önbellekteki $t+1$ satırın hepsini okur.

Karmaşıklık. P_{lin} doğrusal katmanların ağırlık sayısı olsun. N token üretmenin çarpma-toplama (MAC) maliyeti Tablo 3'deki gibidir. Önbelleğin bedeli bellektir: $2LTC$ değer. `gpt-nano` için bu $2 \cdot 3 \cdot 11 \cdot 48 = 3168$ değerdir ve INT8 ile 3 KB tutar.

Tablo 3. Önbelleksiz ve KV önbellekli üretimin maliyeti.

Yöntem	t **adımının maliyeti**	N **adımın toplamı**
`generate()` (önbelleksiz)	$(t{+}1)P_{lin} + O((t{+}1)^2 LC)$	$O(N^2 P_{lin} + N^3 LC)$
KV önbellekli decode	$P_{lin} + 2LC(t{+}1)$	$O(NP_{lin} + N^2 LC)$

Önerme 1 matematiksel bir garantidir. Ek olarak deneysel olarak da doğrulanmıştır: adım adım çalışan tamsayı modelin ürettiği tokenlar, minGPT'nin önbelleksiz `generate()` çıktısıyla 729 girdinin tamamında aynıdır.

8 Parametre ve işlem sayısı analizi

Bir minGPT modelinin parametre sayısı ve token başına işlem sayısı yalnızca (L, C, V, T) boyutlarının kapalı bir fonksiyonudur. Bir bloktaki dört doğrusal katman ve iki LayerNorm şu parametreleri taşır:

$$\underbrace{(3C^2 + 3C)}_{\texttt{c_attn}} + \underbrace{(C^2 + C)}_{\texttt{c_proj}} + \underbrace{(4C^2 + 4C)}_{\texttt{c_fc}} + \underbrace{(4C^2 + C)}_{\texttt{mlp.c_proj}} + \underbrace{4C}_{\mathrm{LN}_1, \mathrm{LN}_2} = 12C^2 + 13C. \tag{19}$$

Bloklara gömme tabloları, son LayerNorm ve çıkış katmanı eklenince:

$$N_{param} = L\,(12C^2 + 13C) + (V + T)\,C + 2C + VC. \tag{20}$$

KV önbellekli decode'da her ağırlık bir token için tam bir kez çarpılır; buna t konumundaki dikkatin iç çarpımları eklenir:

$$\mathrm{MAC}(t) = L \cdot 12C^2 + VC + L \cdot 2C\,(t+1), \qquad \mathrm{FLOP} \approx 2 \cdot \mathrm{MAC}. \tag{21}$$

Tablo 4. `gpt-nano` ($L = 3$, $C = 48$, $V = 3$, $T = 11$) için parametre ve token başına MAC sayıları.

Bileşen	Boyut (çıkış × giriş)	Parametre	MAC / token
`c_attn` (Q, K, V)	144×48	7 056	6 912
`c_proj`	48×48	2 352	2 304
`mlp.c_fc`	192×48	9 408	9 216
`mlp.c_proj`	48×192	9 264	9 216
$\mathrm{LN}_1 + \mathrm{LN}_2$	2×48 (γ, β)	192	–
Blok toplamı		**28 272**	**27 648**
3 blok		84 816	82 944
`wte` + `wpe`	$(3 + 11) \times 48$	672	tablo okuması
`ln_f`	48 (γ, β)	96	–
`lm_head`	3×48	144	144
Dikkat (3 katman)		–	288 $(t + 1)$
Toplam		**85 728**	**83 088 + 288 (t + 1)**

minGPT başlatılırken "number of parameters: 0.09M" yazar; bu sayı `lm_head`'i saymaz (85 584). Adım başına dikkat payı en fazla 3168/86 256 ≈ %3,7'dir. Hesap yükünün %96'dan fazlası doğrusal katmanlardadır; bu gözlem, donanım mimarisinin merkezine bir GEMV motoru konmasının gerekçesidir.

Ölçek karşılaştırması. Aynı formüller GPT-2 small ($L = 12$, $C = 768$, $V = 50\,257$, $T = 1024$) için `lm_head` hariç yaklaşık 124 milyon parametre verir (GPT-2'de `lm_head` token gömmesiyle paylaşılır). Bu model INT8 ile bile yaklaşık 124 MB tutar ve FPGA içindeki SRAM'e sığmaz; harici DRAM ve bant genişliği yönetimi gerektirir. Bu çalışmadaki tasarım ilkeleri aynı kalır; değişen, ağırlıkların nereden ve hangi hızla okunduğudur.

9 Deney görevi: sıralama

Model, minGPT'nin `demo.ipynb` örneğindeki sıralama görevinde eğitilmiştir: {0, 1, 2} rakamlarından oluşan 6 uzunluğundaki bir diziyi küçükten büyüğe sıralamak. Eğitim sonunda model, olası $3^6 = 729$ girdinin tamamını doğru sıralar.

Veri biçimi. Girdi ile çözüm birleştirilir ve modele son token hariç verilir. `0 0 2 1 0 1` girdisi için Tablo 5'taki gibidir. Hedefi −1 olan konumlar kayba katılmaz; model yalnızca sıralı çıktıyı öğrenir. Dizi uzunluğu $2 \cdot 6 - 1 = 11$ olduğu için `block_size` = 11'dir. Örneklerin yarısı tekrar eden rakamı çok olan dizilerden seçilir; girdinin hash değerine göre %25'i test kümesine ayrılır.

Tablo 5. Sıralama görevinin dizi biçimi (girdi `0 0 2 1 0 1`).

Konum	0	1	2	3	4	5	6	7	8	9	10
Girdi x	0	0	2	1	0	1	0	0	0	1	1
Hedef y	−1	−1	−1	−1	−1	**0**	**0**	**0**	**1**	**1**	**2**

Bu görev neden seçildi?

1. **Kapsamlı doğrulama:** girdi uzayı sonlu ve küçüktür. Doğruluk bir örneklemle tahmin edilmez; bütün girdi uzayı üzerinde kesin olarak ölçülür.
2. **Referanssız kontrol:** bir çıktının doğruluğu modelden bağımsız biçimde kontrol edilebilir: çıktı azalmayan olmalı ve girdiyle aynı histograma sahip olmalıdır. Bu özellik Bölüm 25'deki çip üzeri öz-testin temelidir.
3. **Dikkat gerektirir:** çıktının k'inci elemanını üretmek için model girdideki bütün konumlara bakarak her rakamdan kaç tane olduğunu ve kaçının zaten yazıldığını çıkarmalıdır. Bu, tokenlar arası bilgi akışı gerektiren gerçek bir dikkat görevidir.

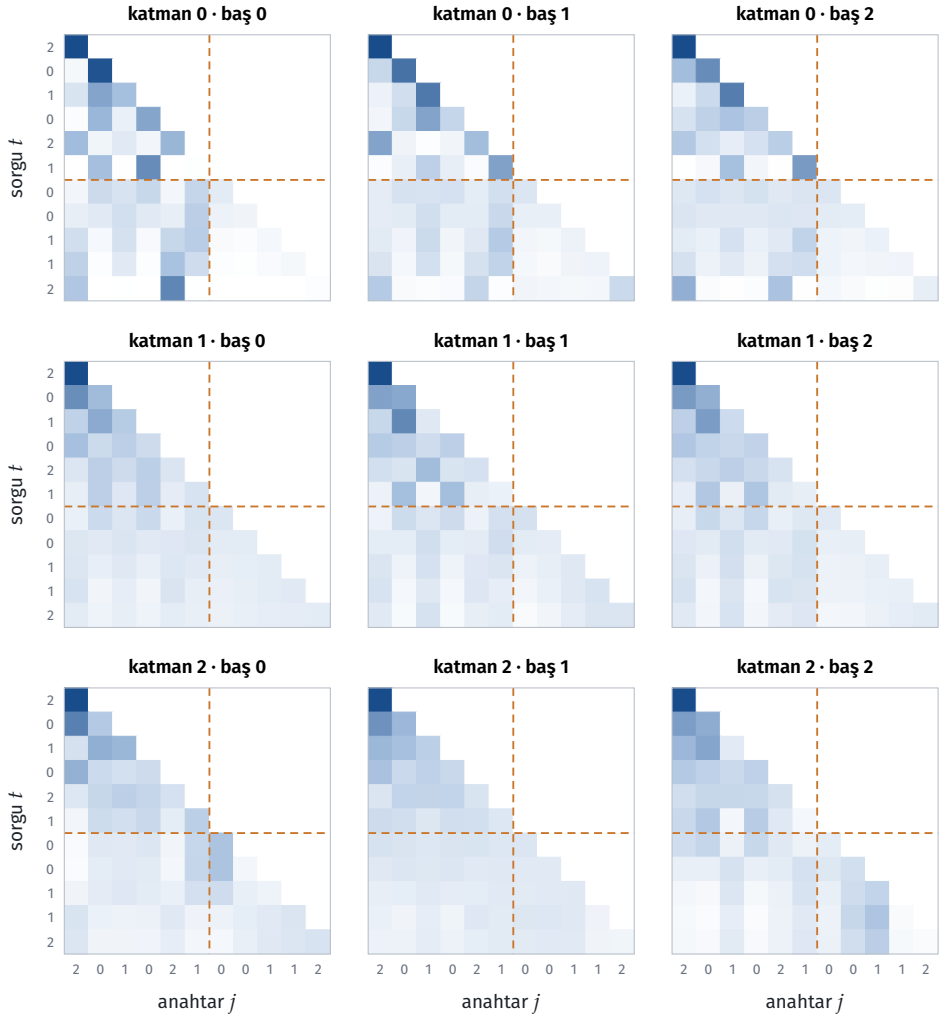
Tablo 6. Model ve eğitim yapılandırması.

Ayar	Değer
Model	`gpt-nano`: $L = 3$, $H = 3$, $C = 48$, $d_h = 16$
Sözlük / bağlam	$V = 3$, $T = 11$
Dropout	0,1 (gömme, dikkat, artık)
İterasyon / batch	5 000 / 64
Optimizasyon	AdamW, $\eta = 5 \times 10^{-4}$, $\beta = (0{,}9;\ 0{,}95)$, $\lambda = 0{,}1$
Tohum	3407
Süre	CPU (Apple M serisi) üzerinde yaklaşık 90 s

Sonuçlar. Başlangıç kaybı 1,089'dur ve rastgele tahminin kaybı olan $\ln 3 = 1{,}0986$'ya çok yakındır. Kayıp 500. iterasyonda 0,0104'e, sonraki iterasyonlarda 10^{-3} mertebesine iner. Açgözlü kod çözmeyle model 729 girdinin **tamamını** doğru sıralar. Eğitilmiş ağırlıklar depoda `model/gpt_sort.pt` dosyasında saklanır; böylece Kısım III'teki bütün adımlar yeniden eğitim yapılmadan tekrarlanabilir.

Model sıralamayı nasıl yapıyor? Dikkat haritaları. Şekil 4, eğitilmiş modelin `2 0 1 0 2 1` girdisini `0 0 1 1 2 2` olarak sıralarken her katmandaki her başın dikkat ağırlıklarını gösterir. Her hücre, t satırındaki sorgunun j sütunundaki anahtara verdiği ağırlıktır; satır toplamı 1'dir. Kesikli çizgiler girdi ile üretilen çıktıyı ayırır.

Haritalardan iki gözlem çıkar. Birincisi, çıktı adımlarında dikkat tek bir konuma odaklanmaz; en büyük ağırlık bu örnekte 0,14 ile 0,64 arasındadır ve ağırlık girdinin geneline yayılır. İkincisi, 729 dizinin tamamında ölçüldüğünde ilk katmanın başları çıktı adımlarında dikkatin %84–92'sini girdi konumlarına verir. Düzgün dağılım bu oran için %77,5 verirdi. Daha derin katmanlarda oran %72–82'ye iner; bu katmanlar daha önce üretilmiş çıktılara daha çok bakar. Bu gözlemler, modelin her rakamın girdideki ve çıktıdaki sayısını karşılaştırarak çalıştığı hipoteziyle uyumludur. Hipotezin kesin olarak doğrulanması ayrı bir yorumlanabilirlik çalışması gerektirir.



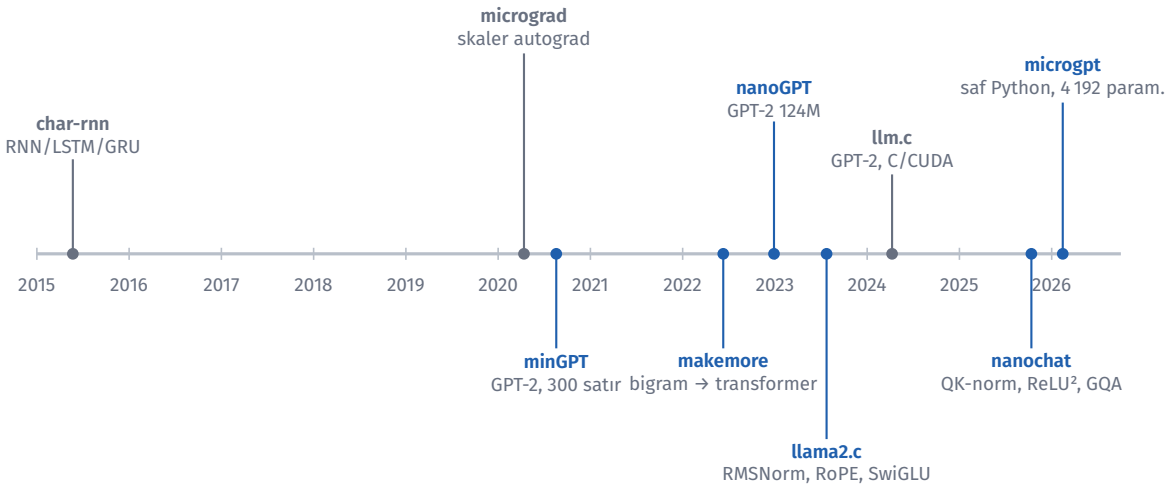
Şekil 4. `2 0 1 0 2 1` girdisi için dokuz dikkat başının ağırlıkları (float model). Eksen etiketleri konumlardaki tokenlardır. Sağ alt çeyrekteki satırlar sıralı çıktının üretildiği adımlardır. Beyaz 0, koyu mavi 1 ağırlığa karşılık gelir.

KISIM II

Karpathy'nin Dil Modeli Ailesi

10 Model ailesine genel bakış

Andrej Karpathy'nin açık kaynak depoları, dil modeli mimarisinin 2003'ten 2026'ya kadarki evrimini adım adım izleyen eğitsel bir seri oluşturur. Her depo bir öncekine tek bir fikir ekler ve hepsi aynı problemi çözer: bir sonraki tokenın olasılığını tahmin etmek. Bu ortak arayüz (Bölüm 1'deki $p_\theta(x_t \mid x_{<t})$), modellerin aynı kayıp, aynı kod çözme yöntemi ve aynı donanım akışıyla karşılaştırılmasını mümkün kılar.



Şekil 5. Karpathy'nin dil modeli depolarının zaman çizelgesi (GitHub oluşturulma tarihleri; microgpt için açıklama yazısının tarihi). Mavi olanlar bu çalışmada ele alınan mimarileri taşır. Gri olanlar ya aynı mimarinin tekrarıdır (llm.c, char-rnn) ya da bir dil modeli değildir (micrograd).

Seriden dokuz mimari seçilmiştir (Tablo 7). makemore'un [29] beş modeli dikkat öncesi dönemi temsil eder. minGPT ve nanoGPT [30] çıkarım matematiği açısından aynı olan GPT-2 mimarisini taşır: nanoGPT'nin farkları isteğe bağlı bias, ağırlık paylaşımı ve Flash Attention'dır;

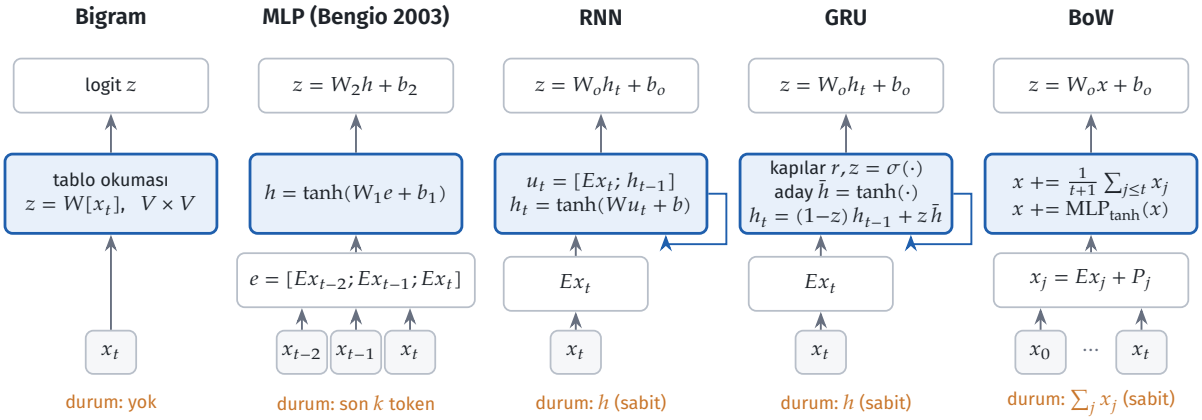
üçü de çıkarımda aynı işlemleri yapar. microgpt (2026) [33] GPT-2'nin sadeleştirilmiş hâlidir. llama2.c [31] Llama 2 [27] mimarisini, nanochat [32] ise 2025 itibarıyla güncel tasarım tercihlerini taşır. llm.c (GPT-2'nin C/CUDA ile eğitimi), char-rnn (RNN/GRU'nun 2015 tarihli Lua sürümü) ve micrograd (skaler otomatik türev motoru) ayrı satır olarak alınmamıştır.

Tablo 7. Ele alınan mimarilerin özellik matrisi. "Durum", çıkarım sırasında geçmişin nasıl saklandığını gösterir.

Model	Kaynak	Geçmişi özetleme	Normalizasyon	Konum	Aktivasyon	Durum
Bigram	makemore	yok (yalnız x_t)	–	–	–	yok
MLP	makemore	sabit pencere	–	pencere sırası	tanh	son k token
RNN	makemore	tekrarlama	–	örtük	tanh	h_t, sabit
GRU	makemore	kapılı tekrarlama	–	örtük	σ, tanh	h_t, sabit
BoW	makemore	nedensel ortalama	–	öğrenilmiş	tanh	$\sum_j x_j$, sabit
GPT-2	minGPT, nanoGPT	çok başlı dikkat	LayerNorm	öğrenilmiş	GELU	KV önbelleği
microgpt	gist (2026)	çok başlı dikkat	RMSNorm, g yok	öğrenilmiş	ReLU	KV önbelleği
Llama 2	llama2.c	GQA	RMSNorm, g var	RoPE	SwiGLU	KV, GQA ile küçük
nanochat	nanochat	GQA + QK-norm	RMSNorm, g yok	RoPE	ReLU^2	KV, GQA ile küçük

11 Dikkat öncesi modeller

makemore'un beş modeli aynı soruyu farklı biçimde yanıtlar: geçmiş tokenlar sabit boyutlu bir vektöre nasıl özetlenir? Şekil 6 beşini aynı görsel dille gösterir. E token gömme tablosudur.



Şekil 6. makemore'daki dikkat öncesi dil modelleri. Mavi kutular geçmişi özetleyen bileşendir; turuncu satır, çıkarımda saklanması gereken durumu gösterir. RNN ve GRU'da geri besleme oku önceki gizli durumu taşır.

Bigram. Model yalnızca son tokena bakar: $p(x_{t+1} = j \mid x_t = i) = \mathrm{softmax}(W_{i,:})_j$, $W \in \mathbb{R}^{V\times V}$. En büyük olabilirlik çözümü kapalı biçimdedir: $W_{ij} = \log N_{ij} + c_i$; burada N_{ij}, eğitim verisinde i'den sonra j gelme sayısıdır. Donanımda tek bir tablo satırı okunur.

MLP (Bengio ve ark., 2003). Bengio ve ark. [19] önerisinde son k tokenın gömmeleri birleştirilir ve iki katmanlı bir ağdan geçirilir:

$$e_t = [\, Ex_{t-k+1}; \ldots; Ex_t \,] \in \mathbb{R}^{kC}, \qquad h_t = \tanh(W_1 e_t + b_1), \qquad z_t = W_2 h_t + b_2. \tag{22}$$

Pencerenin dışında kalan geçmiş tamamen unutulur. makemore, pencere başındaki boş konumlar için ayrı bir dolgu tokenı kullanır.

RNN. Geçmiş, her adımda güncellenen bir gizli durumda özetlenir:

$$h_t = \tanh(W\,[Ex_t; h_{t-1}] + b), \qquad z_t = W_o h_t + b_o. \tag{23}$$

Başlangıç durumu h_0 öğrenilebilir bir parametredir. Bellek, bağlam uzunluğundan bağımsızdır; buna karşılık h_t, h_{t-1} olmadan hesaplanamaz. Ön ek (prompt) bu yüzden paralel işlenemez.

GRU. GRU [20] kapılar kullanır. Kapılar, eski durumun ne kadarının korunacağını öğrenir. $u_t = [Ex_t; h_{t-1}]$ olmak üzere:

$$\begin{aligned} r_t &= \sigma(W_r u_t + b_r), \qquad z_t = \sigma(W_z u_t + b_z), \\ \bar{h}_t &= \tanh(W_h\,[Ex_t; r_t \odot h_{t-1}] + b_h), \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \bar{h}_t. \end{aligned} \tag{24}$$

BoW (nedensel kelime torbası). Geçmişin düzgün ortalaması alınır:

$$y_t = \frac{1}{t+1} \sum_{j \le t} x_j = \sum_{j \le t} \mathrm{softmax}(\mathbf{0})_j \, x_j. \tag{25}$$

Sağdaki eşitlik BoW'un, bütün skorları sıfır olan bir dikkat olduğunu gösterir: dikkatin "içerikten bağımsız" özel durumudur. makemore bunu bir artık blok içinde $x \leftarrow x + \mathrm{BoW}(x)$, $x \leftarrow x + \mathrm{MLP}_{\tanh}(x)$ biçiminde kullanır. Çıkarımda koşan toplam $S_t = S_{t-1} + x_t$ yeterlidir; durum bağlamdan bağımsızdır.

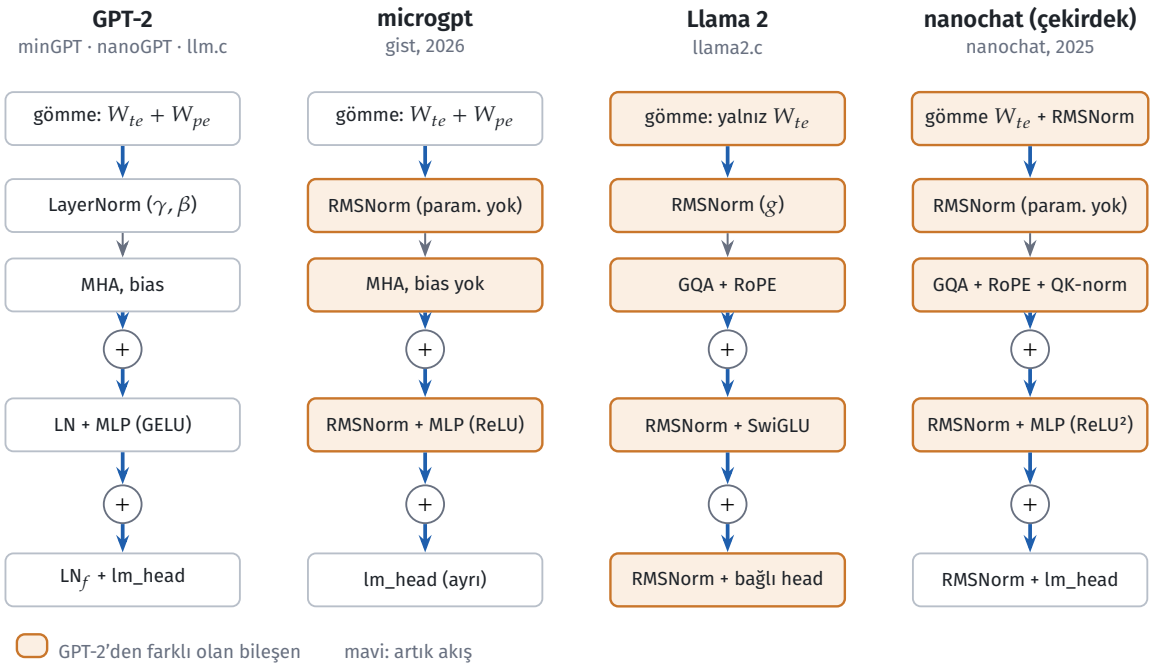
> **Önerme 2** (Tekrarlayan durumun sınırlılığı)**.** RNN'de her $t \ge 1$ için $h_t \in (-1,1)^d$ sağlanır. GRU'da $h_0 \in [-1,1]^d$ ise her t için $h_t \in [-1,1]^d$ sağlanır.

İspat. RNN için tanh'ın değer kümesi $(-1,1)$'dir. GRU için tümevarım: $h_{t-1} \in [-1,1]^d$ ve $\bar{h}_t \in (-1,1)^d$ olsun. $z_t \in (0,1)^d$ olduğundan h_t, her bileşende h_{t-1} ile $\bar{h}_t$'nin dışbükey birleşimidir ve $[-1,1]$ aralığında kalır. □

Bu sonuç, Önerme 6 ve 8'ün tekrarlayan modellerdeki karşılığıdır. Gizli durum kalibrasyon gerekmeden sabit bir biçimle (örneğin INT8, Q0.7) doyma olmadan temsil edilebilir. Tek koşul, eğitim sonunda öğrenilen h_0'ın $[-1,1]$ aralığında olmasıdır; bu kontrol edilmeli, değilse kırpılmalıdır. σ ve tanh ise GELU gibi INT8 girişli 256 elemanlı tablolarla tam olarak hesaplanır.

12 Transformer varyantları

Dört transformer varyantı aynı iskeleti paylaşır: gömme, L adet pre-norm blok, son normalizasyon ve çıkış katmanı. Farklar normalizasyonda, konum kodlamasında, aktivasyonda ve dikkat başlarının paylaşımındadır (Şekil 7).



Şekil 7. Dört transformer varyantında tek bir blok. Turuncu kutular GPT-2'den farklı olan bileşenlerdir. nanochat'in eğitim verimliliği için eklediği ek mekanizmalar (değer gömmeleri, kayan pencere vb.) "çekirdek" sürüme dahil değildir.

12.1 RMSNorm

RMSNorm, LayerNorm'daki ortalama çıkarmayı kaldırır ve yalnızca ölçekler [21]:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2 + \epsilon}, \qquad \mathrm{RMSNorm}(x)_i = g_i \, \frac{x_i}{\mathrm{RMS}(x)}. \tag{26}$$

Llama 2 öğrenilebilir bir kazanç g kullanır; microgpt ve nanochat'te g yoktur. Kazanç, LayerNorm'un γ'sı gibi bir sonraki doğrusal katmana katlanabilir (Bölüm 17).

Önerme 3 (RMSNorm sınırı)**.** $\hat{x} = x/\mathrm{RMS}(x)$ için her bileşende $|\hat{x}_i| \le \sqrt{N}$ sağlanır.

İspat. $x_i^2 \le \sum_j x_j^2 = N\,(\mathrm{RMS}(x)^2 - \epsilon) \le N\,\mathrm{RMS}(x)^2$. □

LayerNorm'daki sınır $\sqrt{N-1}$'di (Önerme 6); RMSNorm'da bir derece serbestlik kaybolmadığı için sınır $\sqrt{N}$'dir. Donanım açısından RMSNorm daha ucuzdur. Toplam s ve fark d_i hesaplanmaz; $Q = \sum x_i^2 + E$ tek geçişte toplanır. Karekök ve bölme aynı kalır.

12.2 Döner konum kodlaması (RoPE)

RoPE [22], konum bilgisini gömmeye eklemek yerine sorgu ve anahtarları konuma bağlı bir açıyla döndürür. q'nun (q_{2i}, q_{2i+1}) çiftleri, m konumunda $m\theta_i$ açısıyla döndürülür:

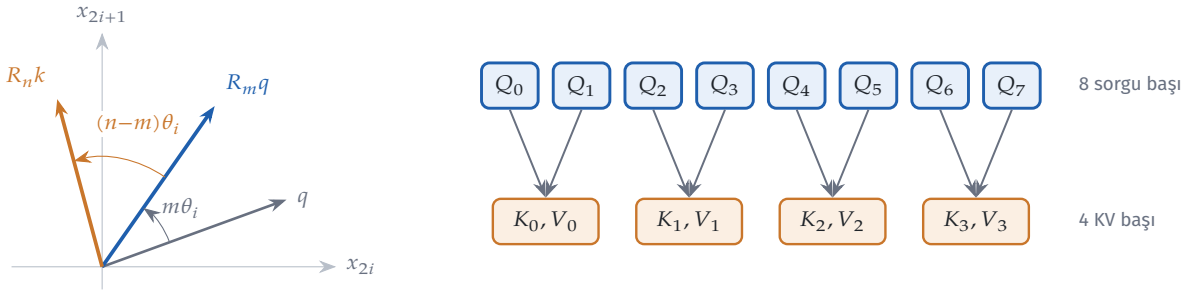
$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}, \qquad \theta_i = 10000^{-2i/d_h}. \tag{27}$$

Bu dönüşümü R_m ile gösterelim.

> **Önerme 4** (Göreli konum). $(R_m q)^\top (R_n k) = q^\top R_{n-m}\, k$. Yani dikkat skoru yalnızca göreli konuma $n - m$ bağlıdır.

İspat. Döndürme matrisleri ortogonaldir ve açıları toplanır: $R_m^\top = R_{-m}$ ve $R_{-m} R_n = R_{n-m}$. □

Donanım için iki önemli özellik vardır. Birincisi, dönme normu korur ($\|R_m q\| = \|q\|$); tamsayı aralığı büyümez. İkincisi, anahtarlar önbelleğe döndürülmüş hâlde yazılır; her adımda yalnızca yeni q_t ve k_t döndürülür. Maliyet çift başına 4 çarpma ve 2 toplamadır. cos ve sin değerleri $T \times d_h/2$ girişlik bir tablodan (ör. Q1.14) okunur. Şekil 8(a) dönmeyi gösterir. llama2.c bitişik çiftleri, nanochat ise vektörün iki yarısını eşler; iki kural bir permütasyon kadar farklıdır.



(a) RoPE **(b) GQA: KV önbelleği yarıya iner**

Şekil 8. (a) RoPE: q ve k konumlarına göre döndürülür; skor yalnızca aradaki açıya, yani göreli konuma bağlıdır. (b) GQA: llama2.c'nin stories260K modelinde 8 sorgu başı 4 KV başını paylaşır; KV önbelleği yarıya iner.

12.3 Aktivasyonlar: ReLU, ReLU^2 ve SwiGLU

microgpt GELU yerine $\mathrm{ReLU}(a) = \max(0, a)$ kullanır; bu tamsayıda tamdır ve tablo bile gerektirmez. nanochat, Primer çalışmasında [26] önerilen $\mathrm{ReLU}(a)^2$'yi kullanır. Kare alma INT8 girişi INT16'ya genişletir, bu yüzden ardından yeniden kuantizasyon gerekir. Llama 2'nin SwiGLU'su [23] ise kapılı bir yapıdır:

$$\mathrm{FFN}(x) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x), \qquad \mathrm{SiLU}(a) = a\,\sigma(a). \tag{28}$$

Üç matris olduğu için ara genişlik $4C$ yerine yaklaşık $\frac{2}{3}\cdot 4C$ seçilir; toplam ağırlık sayısı GELU'lu MLP ile aynı mertebede kalır. Donanımda iki GEMV (W_1, W_3), bir SiLU tablosu ve eleman bazlı bir INT8×INT8 çarpım gerekir.

12.4 Gruplanmış sorgu dikkati (GQA)

GQA [24], H sorgu başının H_{kv} anahtar-değer başını paylaştığı dikkattir. $g = H/H_{kv}$ sorgu başı aynı K ve V'yi kullanır. KV önbelleği $2\,L\,T\,H\,d_h$ yerine $2\,L\,T\,H_{kv}\,d_h$ değer tutar. $H_{kv} = 1$ özel durumuna MQA denir. stories260K'da $H = 8$, $H_{kv} = 4$'tür (Şekil 8(b)).

12.5 QK-norm

QK-norm [25] uygulayan nanochat, skor hesaplanmadan önce her başın q ve k vektörlerini parametresiz RMSNorm'dan geçirir ve her birini $c = 1{,}2$ ile ölçekler.

> **Önerme 5** (QK-norm ile sınırlı skorlar)**.** QK-norm ve $1/\sqrt{d_h}$ ölçeği altında her dikkat skoru $|s| \le c^2\sqrt{d_h}$ sağlar.

İspat. Önerme 3'in ispatından $\|\hat{q}\|^2 = \sum_i \hat{q}_i^2 \le d_h$, benzer şekilde $\|\hat{k}\|^2 \le d_h$ elde edilir. Cauchy–Schwarz ile $|c\hat{q} \cdot c\hat{k}| \le c^2 d_h$, buradan $|s| = |c\hat{q} \cdot c\hat{k}|/\sqrt{d_h} \le c^2\sqrt{d_h}$ bulunur. □

Bu sonucun donanım için önemi şudur: Bölüm 18'teki tamsayı softmax'ın giriş ölçeği $S_q S_k$ kalibrasyonla değil, *analitik olarak* sabitlenebilir. Örneğin $d_h = 16$ için $|s| \le 5{,}76$ olur. Softmax'ın üstel tablosunun kapsaması gereken aralık önceden bilinir.

12.6 nanochat'in ek mekanizmaları ve llama2.c modeli

nanochat'in güncel sürümü, eğitim verimliliği için şu ek mekanizmaları da içerir: alternatif katmanlarda değer gömmeleri (ResFormer), önceki tokenın gömmesini karıştıran "smear", katman başına artık ve x_0 ölçekleri, "backout" çıkarma ve kayan pencereli dikkat. Bunlar hesap yapısını değiştirmez, ama ek tablolar ve skaler karışımlar getirir. Bu çalışmada nanochat'in *çekirdek* mimarisi (Şekil 7) temel alınır; ek mekanizmalar isteğe bağlı ablasyon olarak ele alınabilir.

llama2.c, TinyStories veri kümesinde [28] önceden eğitilmiş hazır modeller sunar. Bunların en küçüğü, bu çalışmanın donanım bütçesine doğrudan uyan **stories260K**'dır (Tablo 8).

Tablo 8. llama2.c stories260K yapılandırması (llama2.c README).

Boyut C	Katman	Sorgu başı	KV başı	Bağlam	Parametre	Doğrulama kaybı
64	5	8	4	512	260 K	1,297

INT8 ile ağırlıklar yaklaşık 260 KB, tam bağlamda KV önbelleği $2 \cdot 5 \cdot 512 \cdot 32 = 163\,840$ değer (160 KB) tutar. Toplam, Zynq-7020'nin yaklaşık 630 KB'lık BRAM'ine sığar. Bu, gerçek bir önceden eğitilmiş modelin kart üzerinde hikâye üretebileceği anlamına gelir.

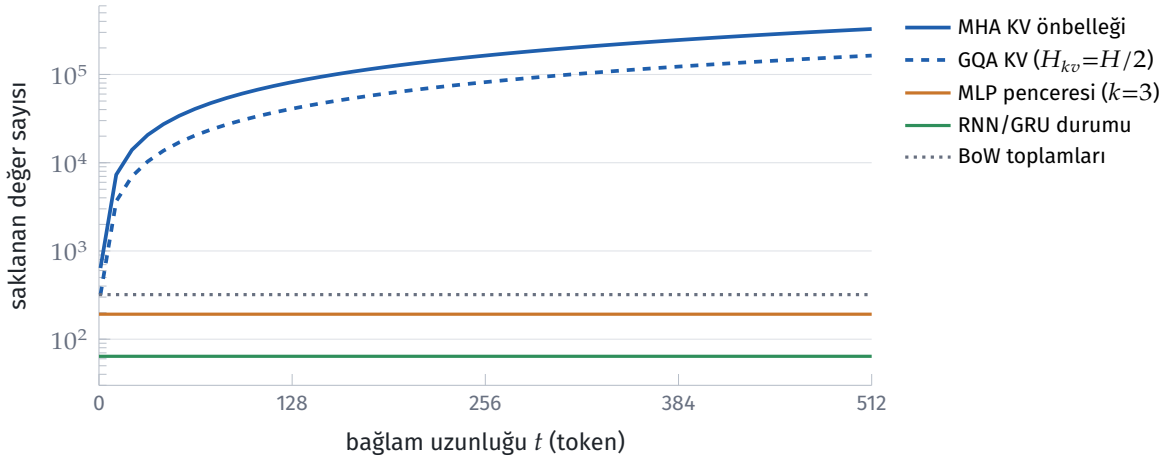
13 Donanım açısından model ailesi

Ailedeki her mimari özellik, küçük bir donanım işlemleri kümesine eşlenir (Tablo 9). Bu işlemlerin çoğu, Kısım III'te minGPT için gerçeklenen birimlerin doğrudan genellemesidir. Bu yüzden ailenin tamamı *tek ve yapılandırılabilir* bir hızlandırıcıyla kapsanabilir.

Tablo 9. Mimari özelliklerin donanım işlemlerine eşlenmesi.

Özellik	Modeller	Tamsayı yöntemi	Garanti
LayerNorm	GPT-2	3 geçiş, karekök, tek bölme (Bölüm 18)	$\lvert\hat{z}\rvert \le \sqrt{N-1}$
RMSNorm	microgpt, Llama, nanochat	2 geçiş, karekök, tek bölme	$\lvert\hat{x}\rvert \le \sqrt{N}$
Öğrenilmiş konum	GPT-2, microgpt, BoW	ROM okuma + INT16 toplama	–
RoPE	Llama, nanochat	cos / sin tablosu (Q1.14), çift başına 4 çarpma	normu korur
GELU, tanh, σ, SiLU	çeşitli	256 girişlik tablo (tam)	–
ReLU / ReLU^2	microgpt / nanochat	$\max(0, x)$ / INT16 kare + requant	–
SwiGLU	Llama	2 GEMV + INT8×INT8 çarpım + requant	–
MHA / GQA	transformerler	KV önbelleği + tamsayı softmax	dışbükeylik
QK-norm	nanochat	q, k için RMSNorm	$\lvert s\rvert \le c^2\sqrt{d_h}$
RNN / GRU hücresi	makemore	ardışık GEMV + kapı tabloları	$\lvert h\rvert \le 1$
Nedensel ortalama	BoW	INT32 koşan toplam + karşılık	dışbükeylik

Bellek ve bağlam. Aileyi ayıran en belirgin donanım özelliği, çıkarım durumunun bağlamla nasıl büyüdüğüdür (Şekil 9). Dikkat tabanlı modellerde KV önbelleği bağlamla doğrusal büyür; GQA bu eğimi H_{kv}/H oranında düşürür. RNN, GRU ve BoW'un durumu sabittir; MLP ise yalnızca son k tokenı tutar. Bunun bedeli de vardır. Sabit durum, geçmişin sıkıştırılması demektir ve modelin kalitesini sınırlar. Bu takası ölçmek, karşılaştırmanın ana sorularından biridir.



Şekil 9. Çıkarım sırasında saklanan değer sayısının bağlam uzunluğuyla değişimi ($L = 5$, $C = 64$, $d_h = 8$; stories260K boyutları). Dikey eksen logaritmiktir. Dikkat tabanlı modellerde bellek doğrusal büyür, tekrarlayan ve ortalama tabanlı modellerde sabit kalır.

Tek mimari, çok model. minGPT hızlandırıcısındaki kontrol FSM'i, `model.py`'nin işlem sırasını sabit olarak kodlar (Bölüm 19). Ailenin tamamı için bu FSM, küçük bir *komut dizisi* (mikrokod) çalıştıran bir denetleyiciyle değiştirilmiştir (Bölüm 27). Python tarafındaki derleyici her model için komut listesini üretir. Donanıma eklenen birimler şunlardır: NORM biriminde RMSNorm ve segment modu, bir RoPE birimi, eleman bazlı işlemler (SwiGLU ve GRU kapıları, BoW ortalaması) ve tekrarlayan durum tamponu. GEMV motoru, yeniden kuantizasyon, tablolar ve doğrulama akışı minGPT'den olduğu gibi alınmıştır.

14 Karşılaştırma yöntemi

Karşılaştırma, minGPT için kurulan akışın (eğitim, katlama, kuantizasyon, tamsayı referans model, RTL, bit-exact doğrulama) her modele aynen uygulanmasıyla yapılmıştır. Bu bölüm yöntemi tanımlar; sonuçlar Kısım IV'tedir.

İki deney hattı.

1. **Hat A: tüm aile, ortak veri.** Dokuz mimarinin hepsi makemore'un `names.txt` veri kümesinde (32 033 isim, $V = 27$, karakter düzeyi) eşleştirilmiş parametre bütçeleriyle eğitilir. Aynı veri ve bütçe, mimariler arasında adil bir karşılaştırma sağlar.
2. **Hat B: gerçek metin üretimi.** Karpathy'nin TinyStories üzerinde hazır eğittiği stories260K (Llama 2 mimarisi, 260 K parametre, 512 token bağlam) aynı akıştan geçirilir. Bu hat, kart üzerindeki hikâye üretimi demosunun temelidir. Diğer transformer varyantlarının aynı ölçekte eğitilip karşılaştırılması gelecek çalışmadır.

Tablo 10. Her model için ölçülecek büyüklükler.

Eksen	Ölçüt
Kalite	test kaybı (nat/token), float ve INT8 için; kuantizasyon kaybı Δ
Boyut	parametre sayısı, INT8 ağırlık belleği
Hesap	token başına MAC; ardışık bağımlılık (paralel ön ek yapılabilir mi?)
Durum	KV önbelleği veya gizli durum büyüklüğü, bağlama göre büyüme
Donanım	çevrim/token, Fmax, LUT/FF/BRAM/DSP, güç, token başına enerji
Referans	aynı karttaki ARM Cortex-A9 üzerinde NumPy ile gecikme; hızlanma

Beklenen katkı. Literatürde transformer hızlandırıcıları genellikle tek bir modelle değerlendirilir. Bu çalışma ise aynı yazarın aynı eğitsel üslupla yazdığı bir model serisini, aynı tamsayı metodolojisi ve aynı donanım üzerinde karşılaştırır. Böylece her mimari fikrin (dikkat, RMSNorm, RoPE, GQA, SwiGLU, QK-norm) donanım maliyeti ile kaliteye katkısı ayrı ayrı görünür hâle gelir.

KISIM III

FPGA İmplementasyonu

15 Tasarım hedefleri ve kısıtları

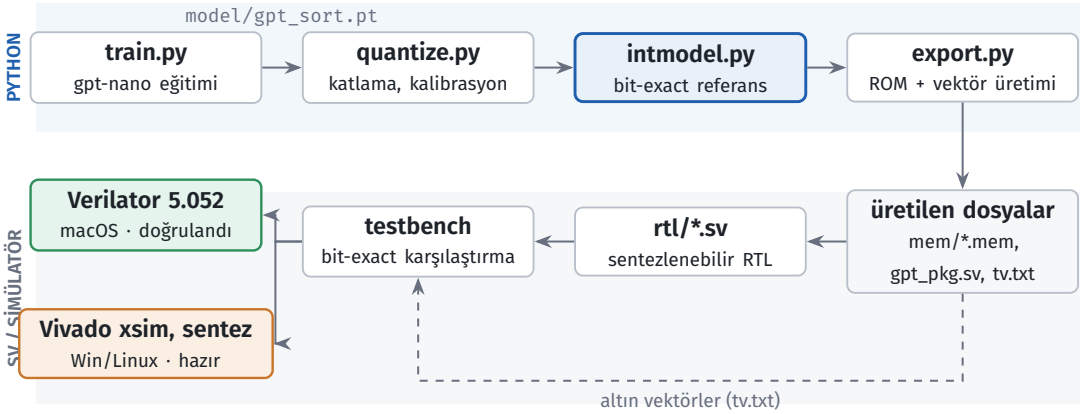
Donanımın amacı, Bölüm 7'deki KV önbellekli decode adımını yalnızca tamsayı aritmetiğiyle ve bütün ağırlıkları çip üzerinde tutarak gerçekleştirmektir. Bu yapılırken bir yazılım referansıyla bit düzeyinde eşdeğerlik korunur.

Hedefler.

1. **Yalnızca tamsayı veri yolu:** kayan nokta birimi kullanılmaz; LayerNorm, softmax ve GELU de tamsayı algoritmalarla hesaplanır.
2. **Çip üzeri bellek:** ağırlıklar BRAM'de, ara değerler dağıtık RAM'de ve yazmaçlarda tutulur; DDR ve DMA gerekmez.
3. **Bit-exact doğrulanabilirlik:** her işlem, Python'daki tamsayı referans modelde aynı bit genişliği, yuvarlama ve doyma kurallarıyla tanımlanır (`sw/intmodel.py`). RTL çıktısı bu modelle tolerans olmadan karşılaştırılır.
4. **Parametriklik:** boyutlar ve sabitler, Python tarafından üretilen bir SystemVerilog paketinden (`rtl/gpt_pkg.sv`) okunur. Farklı bir model yeniden eğitilip dışa aktarıldığında RTL değişmez.
5. **Taşınabilir RTL:** kod sentezlenebilir SystemVerilog alt kümesiyle yazılır; hem Verilator hem de Vivado xsim'de çalışır.

Neden tamsayı? 7-serisi FPGA'larda bir float32 çarpıcı birden fazla DSP48E1 bloğu, ek LUT ve birkaç çevrimlik gecikme gerektirir. INT8×INT8 çarpımı ise tek bir DSP48E1'in 25 × 18 bitlik çarpıcısının küçük bir kısmını kullanır veya doğrudan LUT'larla yapılabilir. Ağırlık belleği float32'ye göre 4 kat küçülür. Ayrıca tamsayı aritmetik deterministiktir: aynı girdi her platformda aynı bitleri verir. Bit-exact doğrulama ancak bu sayede mümkündür.

Yazılımdan donanıma akış. Proje dört Python adımı ve iki simülatör adımından oluşur (Şekil 10). Merkezde, donanımın spesifikasyonu olarak kullanılan tamsayı referans modeli vardır.



Şekil 10. Yazılımdan donanıma akış. Python tarafı modeli eğitir, tamsayıya dönüştürür ve ROM içeriklerini, SystemVerilog paketini ve altın test vektörlerini üretir. RTL, Verilator ile bu Mac'te doğrulanmıştır. Vivado adımları hazırdır, ancak henüz çalıştırılmamıştır.

16 Sayı sistemi ve kuantizasyon teorisi

Model INT8 ağırlık ve aktivasyonlar, INT32 akümülatörler ve INT16 artık akışla temsil edilir. Gerçel ölçekler çalışma zamanında yalnızca tamsayı çarpma ve kaydırma ile uygulanır. Bu biçim, Jacob ve ark.'nın [11] tamsayı-yalnız çıkarım çerçevesini izler.

16.1 Düzgün simetrik kuantizasyon

Gerçel bir r değeri, ölçek $S > 0$ ile b bitlik bir q tamsayısına eşlenir:

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{r}{S}\right), -2^{b-1}, 2^{b-1}-1\right), \qquad \hat{r} = S \cdot q. \tag{29}$$

Aralık içindeki her değer için hata $|r - \hat{r}| \le S/2$'dir. Hata $[-S/2, S/2]$ aralığında düzgün dağılımlı kabul edilirse gürültü gücü $S^2/12$ olur. Her ek bit S'yi yarıya indirir ve sinyal-kuantizasyon gürültüsü oranını (SQNR) yaklaşık $20\log_{10} 2 \approx 6{,}02$ dB artırır [12].

16.2 Taşıyıcı biçimler

- **Ağırlıklar:** INT8, *çıkış kanalı başına* ölçek $S_w[o] = \max_i |W_{oi}|/127$, aralık $[-127, 127]$. Kanal başına ölçek, küçük normlu satırların büyük normlu satırlar yüzünden ezilmesini önler.

- **Aktivasyonlar:** INT8, *tensör başına* sabit ölçek, aralık $[-128, 127]$.
- **Bias:** INT32, ölçeği akümülatörle aynıdır ($S_w[o]\,S_x$); doğrudan akümülatöre eklenebilir.
- **Artık akış:** INT16, ölçeği 2'nin kuvveti olan $S_r = 2^{-FR}$.

16.3 Yalnızca tamsayı GEMV

Bir doğrusal katman $y_o = \sum_i W_{oi}x_i + b_o$ için kuantize temsiller yerine yazılır ve ortak çarpan dışarı alınır:

$$y_o \;\approx\; S_w[o]\,S_x \underbrace{\left(\sum_i q^w_{oi}\,q^x_i \;+\; q^b_o\right)}_{acc_o \in \mathbb{Z}}, \qquad q^b_o = \mathrm{round}\left(\frac{b_o}{S_w[o]\,S_x}\right). \tag{30}$$

Parantez içindeki ifade yalnızca tamsayı çarpma ve toplama içerir; donanımın gerçekten hesapladığı değer budur. Çıktının ölçeği S_y ise $q^y_o = \mathrm{round}(M_o\,acc_o)$ olur. Burada $M_o = S_w[o]\,S_x/S_y$ bir gerçel sayıdır.

16.4 Yeniden kuantizasyon (requantization)

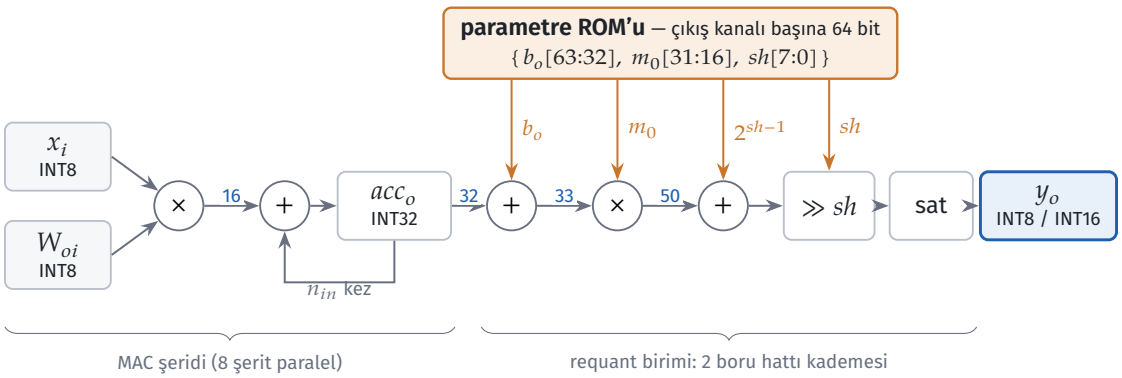
M_o, bir tamsayı çarpan ve bir kaydırma ile temsil edilir:

$$M_o \approx m_0 \cdot 2^{-sh}, \qquad 2^{15} \le m_0 < 2^{16}, \qquad \left|\frac{m_0\,2^{-sh} - M_o}{M_o}\right| \le 2^{-16}. \tag{31}$$

m_0'ın en yüksek bitinin her zaman 1 olması, 16 bitin tamamının hassasiyete harcanmasını sağlar; bu, kayan noktadaki mantis normalizasyonunun tamsayı karşılığıdır. Donanımda yeniden kuantizasyon şu ifadeyle yapılır:

$$q^y_o = \mathrm{sat}_b\left(\left((acc_o + q^b_o)\cdot m_0 + 2^{sh-1}\right) \gg sh\right). \tag{32}$$

Aritmetik sağa kaydırma taban bölmesidir: $a \gg sh = \lfloor a/2^{sh} \rfloor$. Dolayısıyla $(a + 2^{sh-1}) \gg sh = \lfloor a/2^{sh} + \frac{1}{2} \rfloor$, yani yarıyı yukarı yuvarlamadır. sat_b sonucu b bitlik işaretli aralığa kırpar. (b, m_0, sh) üçlüsü her çıkış kanalı için parametre ROM'unda 64 bitlik bir kelime olarak saklanır. Veri yolu Şekil 11'te gösterilmiştir.



Şekil 11. Tamsayı GEMV şeridi ve yeniden kuantizasyon veri yolu. Mavi sayılar bit genişlikleridir. Parametre ROM'u her çıkış kanalı için bias, m_0 ve sh değerlerini sağlar.

Akümülatör genişliği. INT8×INT8 çarpımının büyüklüğü en fazla $127 \cdot 128 = 16\,256$'dır. En geniş giriş $N_{in} = 192$ için $|acc| \le 192 \cdot 16\,256 \approx 3{,}1 \cdot 10^6 < 2^{22}$ olur. 32 bitlik akümülatör taşmaz; gereken en az genişlik işaretiyle 23 bittir.

16.5 Kalibrasyon

Statik ölçekler, float modelin bir kalibrasyon kümesi üzerinde çalıştırılmasıyla belirlenir: her tensörün en büyük mutlak değeri kaydedilir ve $S = \max|r|/127$ alınır. Bu projede kalibrasyon kümesi görevin **bütün girdi uzayıdır** (729 dizi × 11 konum). Bu nedenle çıkarımda kalibre edilmiş aralığın dışında bir aktivasyon oluşamaz; kırpma hatası sıfırdır.

Artık akışın kalibre edilmiş en büyük değeri 5,936'dır. $FR = \lfloor \log_2(32\,767/5{,}936) \rfloor = 12$ seçilir ve akış **Q3.12** biçiminde temsil edilir (aralık ± 8, çözünürlük 2^{-12}). Ölçek 2'nin kuvveti olduğundan gömme toplamı ve artık toplama ek bir ölçek dönüşümü gerektirmez. Katman başına kalibre edilen ölçekler Tablo 11'te verilmiştir.

Tablo 11. Katman başına kalibre edilen aktivasyon ölçekleri. S_q, $1/\sqrt{d_h}$ katlandıktan sonraki sorgu ölçeğidir.

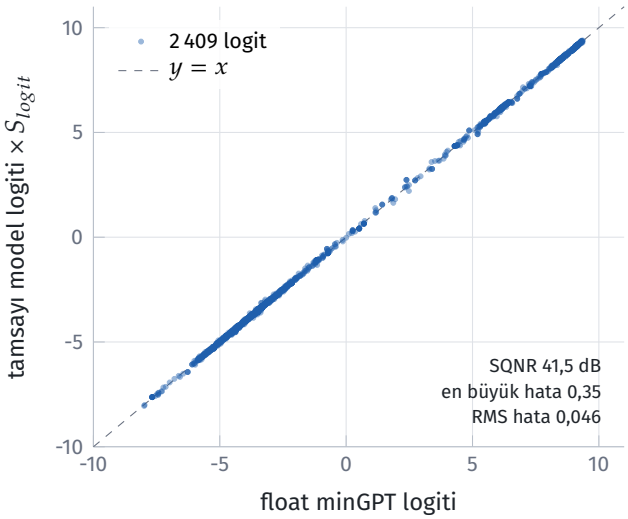
Katman	S_q	S_k	S_v	S_h (GELU giriş)	S_g (GELU çıkış)
0	0,004359	0,01803	0,008157	0,02900	0,02490
1	0,002940	0,008851	0,01075	0,02246	0,01876
2	0,002882	0,01326	0,01076	0,02132	0,02125

16.6 Biçim özeti ve sayısal sonuç

Tablo 12. Tensör biçimleri ve seçim gerekçeleri.

Tensör	Biçim	Ölçek	Gerekçe
Ağırlıklar	INT8	kanal başına	[11]
Bias	INT32	$S_w S_x$	akümülatörle aynı ölçek
Artık akış	INT16	2^{-12} (Q3.12)	2'nin kuvveti; toplama dönüşümsüz
LayerNorm çıktısı	INT8	2^{-4} (Q3.4)	Samuelson sınırı, doyma yok (Önerme 6)
q, k, v	INT8	tensör başına	kalibrasyon
Dikkat çıktısı	INT8	S_v	dışbükeylik, doyma yok (Önerme 8)
GELU giriş/çıkışı	INT8	katman başına	kalibrasyon, tam tablo
Softmax ağırlıkları	UQ1.15	2^{-15}	en büyük terim tam olarak 1
Logitler	INT16	ortak S_{logit}	argmax için ortak ölçek

Tamsayı model, float modelle 729 girdinin tamamında aynı tokenları üretir. Öğretmen zorlamalı logit hatasının en büyüğü 0,35, karekök ortalaması 0,046'dır. Logitlerin ortalamadan sapmasına göre ölçülen SQNR 41,5 dB'dir (Şekil 12).



Şekil 12. Float minGPT logitleri ile tamsayı modelin logitleri (öğretmen zorlamalı, 146 dizi, her ikinci nokta). Noktalar $y = x$ doğrusu üzerinde toplanır.

17 Matematiksel olarak eşdeğer dönüşümler (katlama)

Kuantizasyondan önce, modelin hesapladığı fonksiyonu değiştirmeden donanımı sadeleştiren iki dönüşüm uygulanır. Bu dönüşümler LayerNorm'un afin parametrelerini ve dikkat ölçeğini doğrusal katmanların içine taşır.

LayerNorm afin katlama. Her LayerNorm'un ardından bir doğrusal katman gelir. Bu durumda γ ve β, o katmanın ağırlık ve bias'ına katılabilir:

$$W\,(\gamma \odot \hat{z} + \beta) + b \;=\; \underbrace{(W\,\mathrm{diag}(\gamma))}_{W'}\,\hat{z} \;+\; \underbrace{(W\beta + b)}_{b'}. \tag{33}$$

W', W'nin i'inci sütununun γ_i ile çarpılmasıyla elde edilir. `ln_1` `c_attn`'a, `ln_2` `mlp.c_fc`'ye, `ln_f` de `lm_head`'e katlanır. `lm_head`'in bias'ı yoktur; katlama ona $W\beta_f$ bias'ını ekler. Katlamadan sonra donanımdaki LayerNorm birimi yalnızca $\hat{z}$'yi hesaplar. γ ve β için ne bellek ne çarpıcı gerekir. Bir yan etki olarak $\hat{z}$'nin aralığı artık veriden bağımsız ve matematiksel olarak sınırlıdır. Bu sayede LN çıktısı için kalibrasyon gerektirmeyen sabit bir biçim (Q3.4) kullanılabilir.

Dikkat ölçeği katlama. Skor $(q \cdot k)/\sqrt{d_h} = (q/\sqrt{d_h}) \cdot k$ biçiminde yazılabilir. $1/\sqrt{d_h}$ çarpanı, `c_attn`'ın Q'yu üreten ilk C satırına katlanır:

$$W'_Q = \frac{W_Q\,\mathrm{diag}(\gamma_1)}{\sqrt{d_h}}, \qquad b'_Q = \frac{W_Q\beta_1 + b_Q}{\sqrt{d_h}}, \qquad \sqrt{d_h} = 4. \tag{34}$$

Bundan sonra donanım skorları doğrudan $q' \cdot k$ olarak hesaplar ve bölme yapmaz.

Doğrulama. Katlanmış model float64 ile bağımsız bir NumPy ileri geçişi olarak yazılmıştır. 729 dizinin tümünde her konumdaki logitler, orijinal minGPT çıktısıyla 10^{-4} mutlak tolerans içinde aynıdır. Bu kontrol `quantize.py` her çalıştığında otomatik olarak tekrarlanır.

18 Tamsayı çekirdekler

Doğrusal olmayan üç işlem kayan nokta ve aşkın fonksiyon kullanmadan, yalnızca toplama, çarpma, kaydırma, karşılaştırma, taban bölme ve taban karekökle hesaplanır. Her algoritma için taşma olmadığı ve doymanın gerekmediği ispatlanmıştır.

18.1 Tamsayı LayerNorm

Girdi $N = C = 48$ elemanlı INT16 artık vektörüdür ($S_r = 2^{-12}$). Çıktı Q3.4 biçiminde INT8'dir ($ZF = 4$):

$$\begin{aligned} &s = \textstyle\sum_i x_i, && d_i = Nx_i - s = N\,(x_i - \mu) \quad (\text{tam}),\\ &Q = \textstyle\sum_i d_i^2 + E, && E = \operatorname{round}\left(\epsilon\, N^3/S_r^2\right) \;\Rightarrow\; Q = N^3(\sigma^2+\epsilon)/S_r^2,\\ &rt = \left\lfloor \sqrt{Q} \right\rfloor, && inv = \lfloor K/rt \rfloor, \quad K = \operatorname{round}\left(\sqrt{N}\,2^P\right),\\ &z_i = \operatorname{sat}_8\left(\left(d_i \cdot inv + 2^{P-ZF-1}\right) \gg (P-ZF)\right). \end{aligned} \tag{35}$$

Tanımlar yerine konunca $\hat{z}_i = \sqrt{N}\,d_i/\sqrt{Q}$ olduğu görülür. $inv \approx \sqrt{N}\,2^P/\sqrt{Q}$ olduğundan $d_i \cdot inv/2^{P-ZF} \approx \hat{z}_i\,2^{ZF}$ bulunur; bu, $\hat{z}_i$'nin Q3.4 karşılığıdır. S_r ölçeği yalnızca ϵ terimi E'de görünür; LayerNorm'un ölçek değişmezliği sayesinde başka hiçbir yerde yer almaz. Sabitler: $P = 36$, $E = 18\,554\,259$ ve $K = 476\,102\,500\,705 < 2^{39}$.

> **Önerme 6** (Samuelson eşitsizliği [14]). *N elemanlı her gerçel vektör için $|x_i - \mu| \le \sigma\sqrt{N-1}$ sağlanır.*

İspat. $\sum_j (x_j - \mu) = 0$ olduğundan $(x_i - \mu) = -\sum_{j\neq i}(x_j - \mu)$ yazılabilir. Cauchy–Schwarz eşitsizliğiyle

$$(x_i - \mu)^2 \le (N-1)\sum_{j\neq i}(x_j-\mu)^2 = (N-1)\left(N\sigma^2 - (x_i-\mu)^2\right)$$

bulunur; buradan $N(x_i-\mu)^2 \le N(N-1)\,\sigma^2$ elde edilir. □

Sonuç. $|\hat{z}_i| \le \sqrt{N-1} = \sqrt{47} \approx 6{,}86$ olur; $\epsilon > 0$ bu değeri yalnızca küçültür. Q3.4'ün en büyük değeri $127/16 = 7{,}94$'tür. LayerNorm çıktısı bu yüzden **hiçbir girdi için doymaz**. Bu garanti kalibrasyondan değil matematikten gelir ve $N \le 64$ olan her model için geçerlidir.

> **Önerme 7** (LayerNorm'da taşma yok). *$|d_i \cdot inv| \le K < 2^{39}$.*

İspat. $|d_i| \le \sqrt{\sum_j d_j^2} \le \sqrt{Q}$ sağlanır. d_i tamsayı olduğu için $|d_i| \le \lfloor\sqrt{Q}\rfloor = rt$ olur. Dolayısıyla $|d_i \cdot inv| \le rt \cdot K/rt = K$. □

23 bitlik d_i ile 39 bitlik inv'in çarpımı bu yüzden hiçbir zaman 2^{39}'u geçmez; 64 bitlik işaretli çarpım güvenlidir. $rt \ge \sqrt{E} \approx 4\,307$ olduğundan karekökün taban almasından gelen bağıl hata en fazla $2{,}3 \cdot 10^{-4}$'tür; çıktıdaki $\pm\frac{1}{2}$ LSB yuvarlamasının yanında ihmal edilebilir.

Tek geçişli biçim. $\sum_i d_i^2 = N\,(N\sum_i x_i^2 - s^2)$ özdeşliği tamsayıda *tam* olarak sağlanır. Bu sayede s ve $\sum x^2$ tek geçişte toplanabilir. Kayan noktada bu formun yol açtığı yıkıcı iptal sorunu tamsayı aritmetikte yoktur (gelecek iyileştirme).

18.2 Tamsayı softmax

Bir başın t konumundaki skorları $s_j = q \cdot k_j$ ($j = 0..t$) INT32'dir ve ölçekleri $S_s = S_q S_k$'dir. Hesap dört adımda yapılır (Şekil 13).

(i) Kaydırma. $m = \max_j s_j$ alınır. Kaydırma değişmezliği sayesinde bütün üsler ≤ 0 olur ve $e_j \in (0, 1]$ aralığında kalır.

(ii) 2 tabanına geçiş. $e^x = 2^{x \log_2 e}$ eşitliği kullanılır. Skor farkı, normalize edilmiş bir tamsayı sabitle çarpılarak 2 tabanında sıfır veya pozitif bir üsse dönüştürülür:

$$u_j = (m - s_j) \cdot C_{exp}, \qquad C_{exp}\, 2^{-F} \approx S_q S_k \log_2 e, \qquad 2^{15} \le C_{exp} < 2^{16}. \tag{36}$$

Katman başına (C_{exp}, F) değerleri $(60\,882;\ 29)$, $(40\,315;\ 30)$ ve $(59\,216;\ 30)$'dur. Bu durumda $e_j = 2^{-u_j / 2^F}$ olur.

(iii) Aralık indirgeme ve tablo. u_j, tam kısım n ve kesir kısmı olarak ayrılır. Tam kısım bir sağa kaydırmaya, kesrin üst 8 biti 256 girişlik bir tabloya dönüşür:

$$n = u_j \gg F, \qquad k = (u_j \gg (F-8)) \,\&\, 255, \qquad \mathrm{LUT}[k] = \mathrm{round}\left(2^{15} \cdot 2^{-k/256}\right),$$
$$e_j = \begin{cases} \mathrm{LUT}[k] \gg n, & n \le 15, \\ 0, & n > 15. \end{cases} \tag{37}$$

e_j UQ1.15 biçimindedir ve en büyük skor için tam olarak 2^{15} $(= 1{,}0)$ değerini alır.

Hata analizi. Kesrin üst 8 bite kesilmesi üste 2^{-8}'den küçük bir hata ekler. Bu, e_j'de $2^{2^{-8}} - 1 \approx$ %0,27'den küçük bir bağıl hata demektir. Bu hata bütün terimleri aynı yönde etkilediği için E'ye bölünürken kısmen birbirini götürür. $\gg n$ kaydırmasının kesmesi en fazla 2^{-15} mutlak hata ekler. $n > 15$ için sıfır alınan terimler en büyük terimin 2^{-16}'sından küçüktür.

(iv) Ertelenmiş normalizasyon. Her olasılığı ayrı ayrı bölmek yerine önce ağırlıklı toplam alınır; bölme sonda tek bir karşılık (reciprocal) ile yapılır:

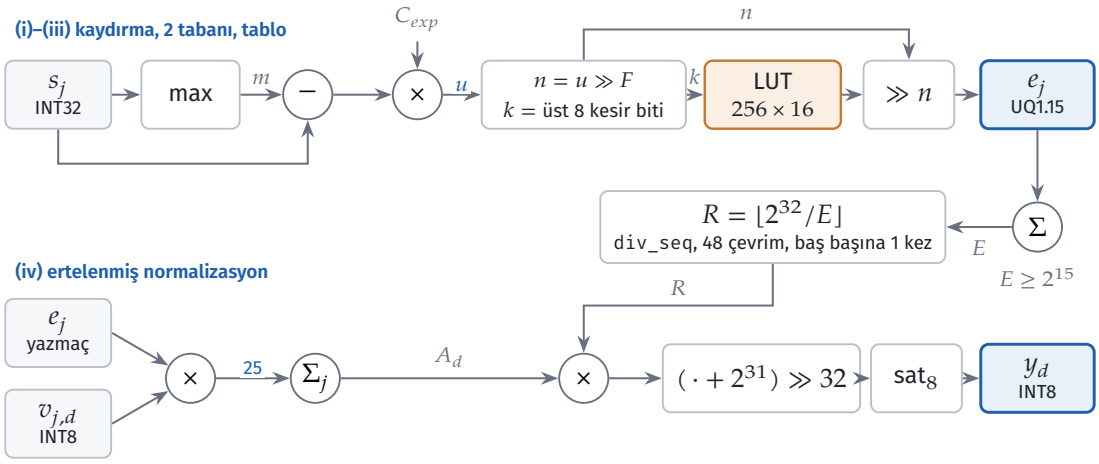
$$A_d = \sum_{j=0}^{t} e_j\, v_{j,d}, \qquad E = \sum_{j=0}^{t} e_j, \qquad R = \left\lfloor \frac{2^{32}}{E} \right\rfloor, \qquad y_d = \mathrm{sat}_8\left(\left(A_d R + 2^{31}\right) \gg 32\right). \tag{38}$$

Böylece baş başına **tek bir bölme** yapılır. En büyük terim için $e = 2^{15}$ olduğundan $E \ge 2^{15}$ olur; sıfıra bölme imkânsızdır ve $R \le 2^{17}$'dir.

> **Önerme 8** (Dışbükeylik ve doymazlık)**.** Dikkat çıktısı y_d, ölçeği $S_y = S_v$ seçildiğinde hiçbir girdi için doymaz.

İspat. $p_j = e_j / E \ge 0$ ve $\sum_j p_j = 1$ olduğundan y_d, $v_{j,d}$ değerlerinin dışbükey birleşimidir; $\min_j v_{j,d} \le y_d \le \max_j v_{j,d}$ sağlanır. Tamsayı tarafta R taban alınarak küçüldüğü için $|A_d R / 2^{32}| \le |A_d| / E \le 127$ olur. □

Bu yöntem, I-BERT'in [13] polinom tabanlı `i-exp` fonksiyonuna bir alternatiftir. FPGA'da küçük LUT-ROM'lar ucuz olduğu için tablo tercih edilmiştir.



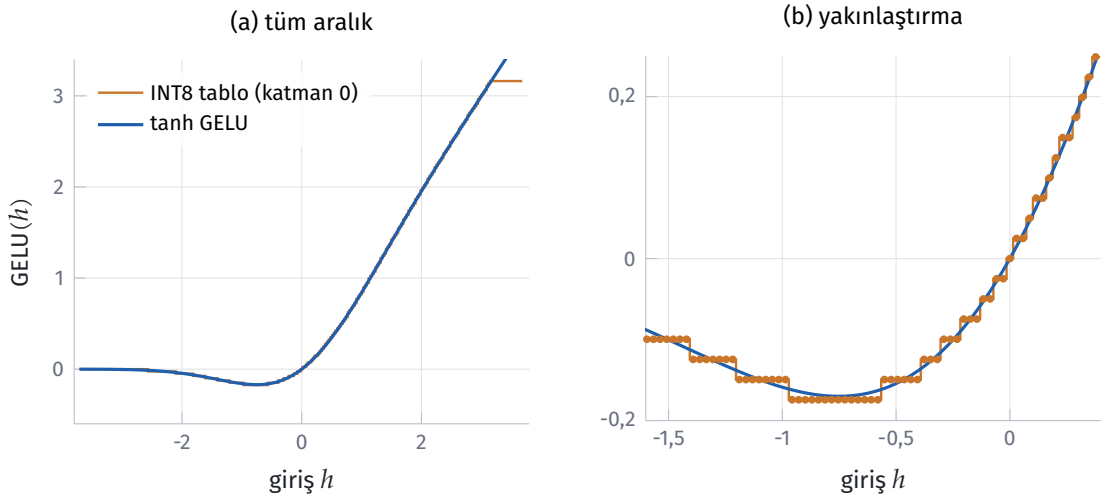
Şekil 13. Tamsayı softmax ve ertelenmiş normalizasyon. Üstel ağırlıklar bir çarpma, bir tablo ve bir kaydırmayla hesaplanır. Baş başına tek bir bölme yapılır.

18.3 GELU: tam tablo

`c_fc` çıktısı INT8'e (S_h) yeniden kuantize edilir. Bu yüzden GELU'nun girdisi yalnızca 256 farklı değer alabilir ve fonksiyon bir tabloyla *tam olarak* temsil edilir:

$$\text{GELU_LUT}_\ell[q] = \text{sat}_8\left(\text{round}\left(\text{GELU}(q \cdot S_h)/S_g\right)\right), \qquad q \in \{-128, \ldots, 127\}. \tag{39}$$

Tablo, minGPT'nin tanh tabanlı GELU'sunu birebir kullanır. Hata yalnızca giriş ve çıkış kuantizasyonundan gelir; bir fonksiyon yaklaşımı yoktur (Şekil 14). Her katmanın kendi tablosu vardır (3×256 bayt). Tablo, INT8 değerin iki tümleyenli bitleriyle doğrudan adreslenir: adres $= \{\ell, h[7{:}0]\}$.



Şekil 14. Katman 0'ın gerçek GELU tablosu (turuncu) ve minGPT'nin tanh GELU'su (mavi). (b) Yakınlaştırmada INT8 adımları görülür. (a)'nın sağ ucundaki düz kısım çıkış doymasıdır: S_g en büyük *gözlenen* çıkıştan kalibre edildiği için yalnızca kalibrasyonda hiç görülmeyen pozitif girişlerde oluşur.

18.4 Argmax ve artık toplama

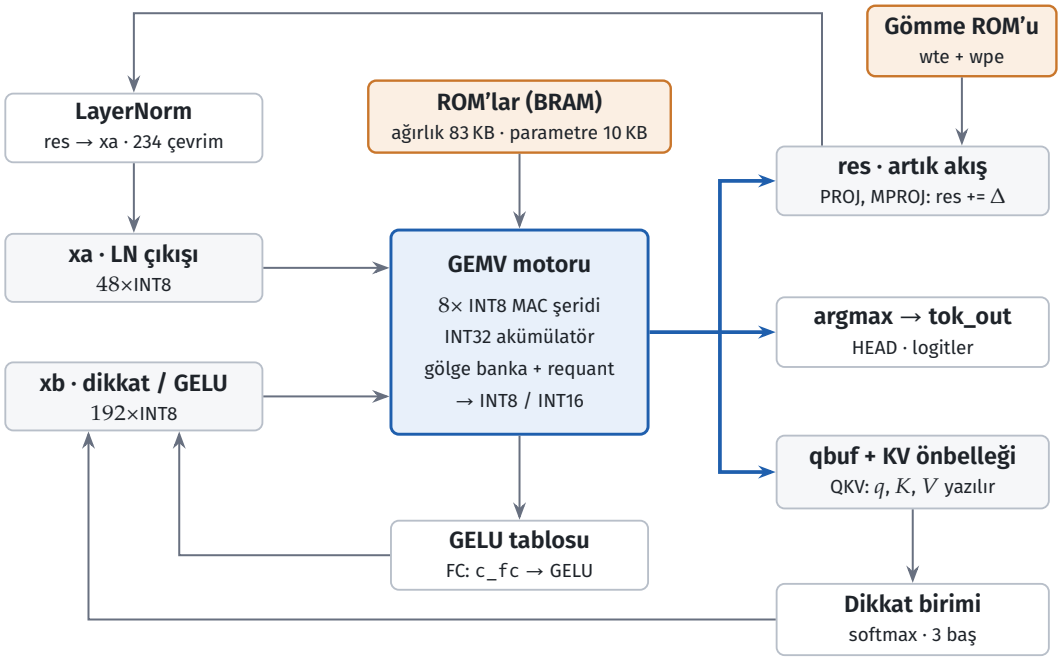
`lm_head` çıktıları ortak bir ölçeğe $(S_{logit}$, INT16) yeniden kuantize edilir. Kanal başına ölçek farkları yeniden kuantizasyonda yok olur; tamsayı karşılaştırma bu yüzden doğrudur. Eşitlik

durumunda en küçük indeks seçilir ve referans model de aynı kuralı uygular. GEMV motoru 8'in katı çıkış istediği için `lm_head` 8 satıra sıfır ağırlıklarla doldurulur; fazla satırlar argmax'a katılmaz.

Artık toplama, `c_proj` ve `mlp.c_proj` çıktılarını doğrudan S_r ölçeğinde INT16'ya yeniden kuantize eder ve $r \leftarrow \mathrm{sat}_{16}(r+\Delta)$ uygular. Gömme adımı da aynı biçimdedir: $x^{(0)} = \mathrm{sat}_{16}(W_{te}[\tau] + W_{pe}[t])$.

19 Donanım mimarisi

Hızlandırıcı, bir token için minGPT'nin bütün ileri geçişini katman katman ve sırayla yürütür. Bütün doğrusal katmanlar aynı GEMV motorunu paylaşır; LayerNorm ve dikkatin kendi ardışık birimleri vardır (Şekil 15). GEMV motoru girdisini LN tamponundan (xa) veya dikkat/GELU tamponundan (xb) alır. Çıktısını o anki duruma göre dört hedeften birine yazar. Artık akış (res) ve KV önbelleği, bloklar ve tokenlar arasında durumu taşıyan iki bellektir.



Şekil 15. `gpt_top` veri yolu. Turuncu kutular ROM'lardır, gri kutular tamponlardır. Mavi oklar GEMV çıkış akışının olası hedefleridir.

19.1 Üst modül arayüzü

Bir dizi üretmek için `pos` $= 0, 1, 2, \ldots$ sırayla verilir. KV önbelleği konumlar arasında korunur.

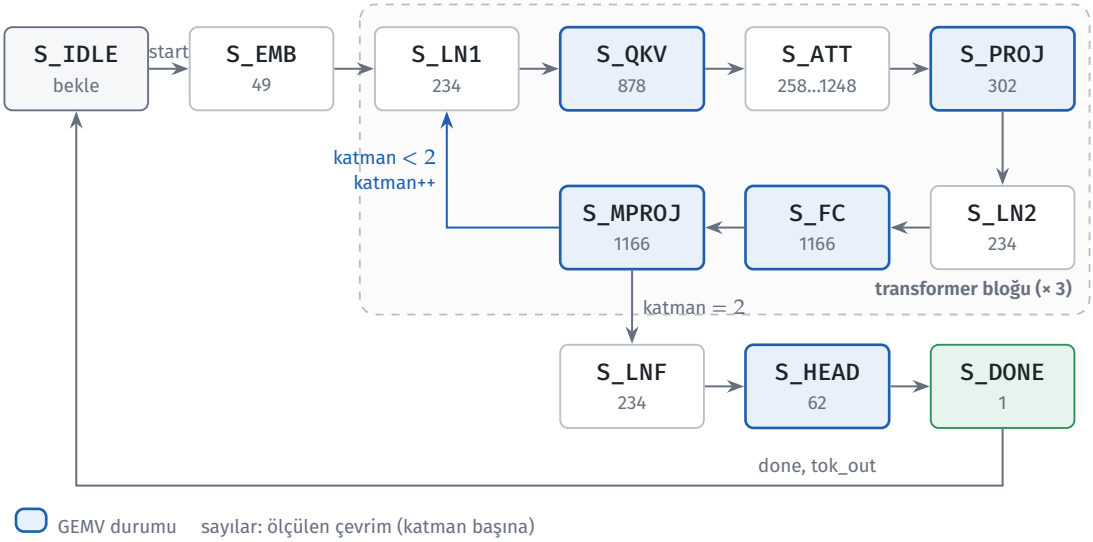
19.2 Kontrol FSM'i

FSM, `model.py`'deki `forward()` fonksiyonunu satır satır izler (Şekil 16). Her durumun ilk çevriminde (`kick`) ilgili birim başlatılır; durum, birimin `done` darbesini bekler. `S_LN1`'den

Tablo 13. `rtl/gpt_top.sv` portları.

Port	Yön	Açıklama
`clk`, `rst_n`	giriş	saat, senkron aktif-düşük reset
`start`	giriş	1 çevrimlik darbe: `tok_in` tokenını `pos` konumunda işle
`tok_in`, `pos`	giriş	token (2 bit) ve konum (4 bit, $0 \le$ `pos` < 11)
`busy`, `done`	çıkış	meşgul / bitti (1 çevrimlik darbe)
`tok_out`	çıkış	en büyük logitin indeksi, yani sonraki token
`logits`	çıkış	3×INT16 logit
`dbg_addr`, `dbg_res`	giriş/çıkış	artık tamponunun asenkron okunması (test için)

`S_MPROJ`'a kadar olan yedi durum her katman için tekrarlanır. GEMV'nin ağırlık taban adresi katman sayacından hesaplanır: `layer`·`W_PER_LAYER`+`W_OFF_*`.



Şekil 16. Kontrol FSM'i. Mavi durumlar GEMV motorunu kullanır. Sayılar katman başına ölçülen çevrimlerdir; dikkat süresi konuma (t) bağlıdır.

Tasarım kararları.

- **Zaman çoklamalı birimler:** tek GEMV motoru ve tek LN birimi bütün katmanlar için sırayla kullanılır; alan küçük tutulur.
- **Akış hâlinde çıktı:** GEMV çıktısı (indeks, değer) çiftleri olarak akar. Hedef tampon FSM durumundan seçildiği için ara tampon gerekmez.
- **Kaynaştırılmış GELU:** GELU tablosu FC çıktısının yazma yoluna yerleştirilmiştir; ayrı bir aşama yoktur.
- **Akış hâlinde argmax:** HEAD çıktıları akarken en büyük logit güncellenir; softmax hesaplanmaz.

Tablo 14. FSM durumları ve `model.py` karşılıkları.

Durum	Birim	Okur → yazar	`model.py`
`S_EMB`	gömme ROM'u	wte, wpe → res	`wte(idx) + wpe(pos)`
`S_LN1`	layernorm	res → xa	`ln_1(x)`
`S_QKV`	gemv 48 → 144	xa → qbuf, K, V	`c_attn`
`S_ATT`	attention	qbuf, K, V → xb	`softmax(q k`$^{\top}$`) v`
`S_PROJ`	gemv 48 → 48	xb → res (+=)	`x + c_proj(y)`
`S_LN2`	layernorm	res → xa	`ln_2(x)`
`S_FC`	gemv 48 → 192 + GELU	xa → xb	`gelu(c_fc(·))`
`S_MPROJ`	gemv 192 → 48	xb → res (+=)	`x + mlp.c_proj(·)`
`S_LNF`	layernorm	res → xa	`ln_f(x)`
`S_HEAD`	gemv 48 → 8 + argmax	xa → tok_out	`lm_head` + argmax

20 GEMV motoru

Token başına hesabın %96'sından fazlası matris-vektör çarpımı olduğu için tasarımın merkezinde $P = 8$ şeritli, **çıkış-durgun** (*output-stationary*) bir GEMV motoru vardır (Şekil 17). Motor, ölçülen 10 598 çevrimin 10 416'sında faydalı iş yapar; verimi %98,3'tür.

20.1 Dataflow seçimi

Sze ve ark. [15] hızlandırıcıları hangi verinin işlem biriminde sabit kaldığına göre sınıflandırır: ağırlık-durgun, girdi-durgun ve çıkış-durgun. Decode sırasındaki GEMV için çıkış-durgun yapının gerekçesi şudur:

- **Ağırlıklar yeniden kullanılmaz:** her ağırlık bir token için tam bir kez çarpılır; ağırlıkları işlem biriminde tutmak bir şey kazandırmaz.
- **Girdi P kez yeniden kullanılır:** aynı x_i, P farklı çıkış nöronu tarafından kullanılır; bütün şeritlere yayınlanır.
- **Kısmi toplamlar yerinde kalır:** her şeridin INT32 akümülatörü n_{in} çevrim boyunca kendi yazmacında birikir.

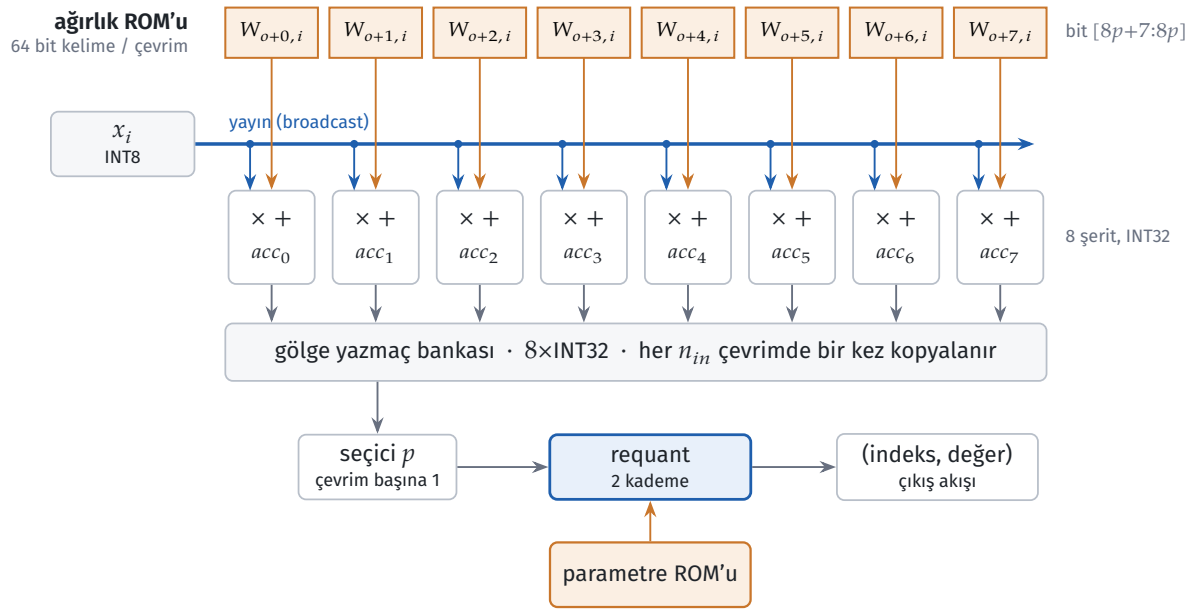
20.2 Ağırlık ROM düzeni

ROM kelimesi, $P = 8$ çıkışın aynı sütundaki ağırlıklarını taşır; şerit p kelimenin $[8p{+}7 : 8p]$ bitlerini kullanır:

$$\text{word}(o_b, i) = \{\, W_{o_b P+7,\, i},\ \ldots,\ W_{o_b P,\, i} \,\}, \qquad acc_p \mathrel{+}= W_{o_b P+p,\, i} \cdot x_i \quad (p = 0..7). \tag{40}$$

Her doğrusal katman için önce çıkış bloğu o_b, sonra giriş indeksi i sırasıyla yazılır. Adres bu yüzden çağrı boyunca birer birer artan bir sayaçtır:

$$\text{addr} = \ell \cdot 3456 + \text{OFF}_{lin} + o_b \cdot n_{in} + i, \qquad \text{OFF} = \{\texttt{attn}{:}\,0,\ \texttt{proj}{:}\,864,\ \texttt{fc}{:}\,1152,\ \texttt{mproj}{:}\,2304\}. \tag{41}$$



Şekil 17. GEMV motoru. Her çevrimde tek bir x_i sekiz şeride yayınlanır ve ağırlık ROM'undan sekiz çıkışın aynı sütundaki ağırlıklarını taşıyan 64 bitlik tek bir kelime okunur. Akümülatörler bir gölge bankaya kopyalanır ve tek bir requant birimine sırayla aktarılır.

`lm_head` 10 368 adresinden başlar; ROM'un toplam derinliği 10 416 kelimedir (83 328 bayt). Parametre ROM'u aynı sırayı çıkış kanalı başına bir kelimeyle izler (1 304 kelime). MAC döngüsünün RTL'deki karşılığı şudur:

```
always_comb begin
  for (int p = 0; p < P; p++) begin
    prod[p]   = $signed(w_word[8*p +: 8]) * x1;        // INT8 x INT8 -> INT16
    acc_nx[p] = (first1 ? 32'sd0 : acc[p]) + prod[p];  // INT32 birikim
  end
end
```

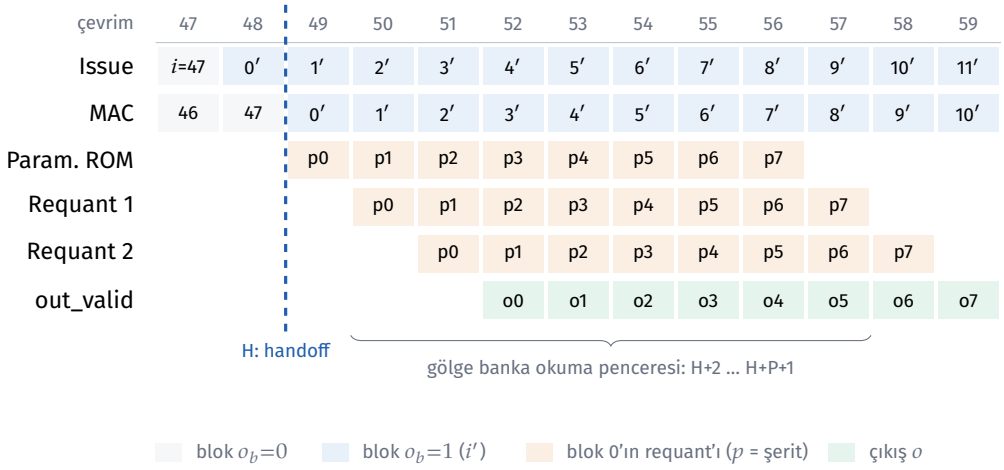
Bloğun ilk elemanında akümülatör sıfır yerine doğrudan çarpımla yüklenir (`first1`); ayrı bir temizleme çevrimi gerekmez.

20.3 Boru hattı ve gölge banka

Motor üç aşamalıdır: *issue* (ROM ve aktivasyon adresi), *MAC* (ROM verisi geçerli, 8 şerit biriktirir) ve *requant*. Bloğun son elemanında 8 akümülatör gölge bankaya kopyalanır (*handoff*). Gölge banka her çevrimde bir değer olmak üzere requant birimine aktarılır; bu sırada şeritler bir sonraki 8 çıkışı hesaplamaya başlamıştır. Şekil 18 zamanlamayı gösterir.

> **Önerme 9** (Gölge banka güvenliği)**.** $n_{in} \geq P + 1$ ise gölge bankadaki hiçbir değerin üzerine okunmadan yazılmaz.

İspat. H çevriminde yazılan gölge banka H+2 ...H+P + 1 çevrimlerinde okunur. Bir sonraki handoff H+n_{in} çevriminin sonundadır. H+P + 1 ≤ H+n_{in} koşulu $n_{in} \geq P + 1$'e denktir. Modeldeki en küçük n_{in} 48'dir; RTL koşulu simülasyonda bir `assert` ile denetler. □



Şekil 18. GEMV boru hattı zamanlaması ($n_{in} = 48$, $P = 8$). Blok 0'ın son MAC'i 48. çevrimdedir; akümülatörler bu çevrimin sonunda gölge bankaya kopyalanır (H). Blok 1'in MAC'leri boşluksuz devam ederken blok 0'ın sekiz çıkışı 52–59. çevrimlerde akar.

20.4 Requant birimi

Bölüm 16'teki ifade iki boru hattı aşamasında hesaplanır: önce 33 bitlik $(acc+b)$ toplamı işaretli 17 bitlik $\{0, m_0\}$ ile çarpılır ve 64 bitlik yazmaca alınır; sonra 2^{sh-1} eklenir, aritmetik kaydırma yapılır ve sonuç `out16` bayrağına göre INT8 veya INT16 aralığına doyurulur:

```
always_ff @(posedge clk) prod_r <= sum * m0_s;              // kademe 1
assign shifted = (prod_r + (64'sd1 <<< (sh_r - 6'd1))) >>> sh_r;
always_ff @(posedge clk) out <= sat(shifted, out16_r);      // kademe 2
```

20.5 Çevrim sayısı

Bir GEMV çağrısı $(n_{out}/P) \cdot n_{in} + 14$ çevrim sürer. Ek 14 çevrim son bloğun yeniden kuantizasyonundan, boru hattının dolup boşalmasından ve FSM geçişinden gelir.

Tablo 15. GEMV çağrılarının ideal ve ölçülen çevrim sayıları.

Katman	$n_{out} \times n_{in}$	İdeal	Ölçülen
`c_attn`	144×48	864	878
`c_proj`	48×48	288	302
`mlp.c_fc`	192×48	1 152	1 166
`mlp.c_proj`	48×192	1 152	1 166
`lm_head`	8×48	48	62

Tüm token için ideal toplam $3 \cdot 3456 + 48 = 10\,416$ çevrimdir; bu, ağırlık ROM'unun derinliğine eşittir, çünkü motor her çevrimde bir ROM kelimesi tüketir.

21 LayerNorm ve dikkat birimleri

LayerNorm ve dikkat hesap yükünün küçük bir kısmını oluşturur. Bu yüzden ikisi de çevrim başına bir işlem yapan ardışık birimler olarak tasarlanmıştır: LN başına 234 çevrim, dikkat için katman başına ortalama 752 çevrim.

21.1 Ardışık karekök

Taban karekök, elle karekök almaya benzeyen *basamak basamak* (*digit-by-digit*) yöntemle hesaplanır. Her çevrimde girdiden 2 bit tüketilir ve köke 1 bit eklenir; 64 bitlik girdi için 32 çevrim sürer:

$$rem \leftarrow 4\,rem + (\text{sıradaki 2 bit}), \qquad trial = 4\,root + 1,$$
$$(rem,\ root) \leftarrow \begin{cases} (rem - trial,\ 2\,root + 1), & rem \geq trial, \\ (rem,\ 2\,root), & \text{aksi hâlde.} \end{cases} \tag{42}$$

Değişmez: i adımdan sonra N_i girdinin en üst $2i$ biti olsun. $root_i = \lfloor \sqrt{N_i} \rfloor$ ve $rem_i = N_i - root_i^2 \leq 2\,root_i$ sağlanır. Aday kök $2r + 1$'dir ve $(2r+1)^2 = 4r^2 + 4r + 1$ olduğundan karşılaştırılacak değer tam olarak $4r + 1$'dir.

21.2 Ardışık bölücü

Taban bölme, işaretsiz *geri yüklemeli* (*restoring*) yöntemle her çevrimde bir bölüm biti üretir (48 bit için 48 çevrim): $r \leftarrow 2r + \text{bit}$; $r \geq D$ ise $r \leftarrow r - D$ ve bölüm biti 1. Değişmez: $q_i = \lfloor N_i / D \rfloor$, $0 \leq r_i < D$. Bölücü LayerNorm'da K/rt, softmax'ta $2^{32}/E$ için kullanılır ve her ikisinde de yalnızca *bir kez* çalışır. Her iki birim, birim testinde 2 000 rastgele girdiyle referans gerektirmeyen koşullara karşı doğrulanmıştır: $r^2 \leq n < (r+1)^2$ ve $q\,d \leq n < (q+1)\,d$.

21.3 LayerNorm birimi

Tablo 16. LayerNorm biriminin aşamaları (toplam 234 çevrim, ölçülen).

Aşama	İşlem	Çevrim
SUM	$s = \sum x_i$	48
VAR	$d_i = 48x_i - s$ (yazmaçlanır); $Q \mathrel{+}= d_i^2$	$48 + 1$
SQRT	$rt = \lfloor \sqrt{Q + E} \rfloor$	$32 + 1$
DIV	$inv = \lfloor K/rt \rfloor$	$48 + 1$
OUT	$z_i = \mathrm{sat}_8((d_i\,inv + 2^{31}) \gg 32)$	$48 + 2$

d_i iki kez hesaplanır (VAR ve OUT); böylece 48 elemanlı bir d tamponuna gerek kalmaz. Bit genişlikleri ihtiyacın üstünde seçilmiştir: d 32 bit (gereken 23), Q 64 bit (gereken yaklaşık 50), inv 48 bit, $d \cdot inv$ 64 bit (Önerme 7'ye göre 2^{39} ile sınırlı).

21.4 Dikkat birimi

Dikkat birimi, t konumundaki tek sorgu için üç başı sırayla işler. Önbellek adreslemesi doğrusaldır:

$$\mathrm{addr}_{KV}(\ell, j, h, d) = (\ell\, T + j)\, C + h\, d_h + d, \qquad 0 \le \mathrm{addr}_{KV} < L\, T\, C = 1584. \tag{43}$$

QKV GEMV'si (ℓ, t) satırını yazar, dikkat $(\ell, 0..t)$ satırlarını okur. Yeni bir diziye $t = 0$ ile başlanırsa eski satırlar silinmez; $j > t$ satırları hiç okunmadığı için bu zararsızdır.

Tablo 17. Dikkat biriminin baş başına aşamaları.

Aşama	İşlem	Çevrim
`SCORE`	$s_j = \sum_d q_d\, k_{j,d}$ (INT8×INT8), koşan maksimum m	$16(t+1)$
`EXP`	kademe 1: $u_j = (m - s_j)\, C_{exp}$; kademe 2: $e_j = \mathrm{LUT}[k] \gg n$, $E \mathrel{+}= e_j$	$(t+1)+2$
`DIV`	$R = \lfloor 2^{32}/E \rfloor$	$48+2$
`AV`	$A_d = \sum_j e_j\, v_{j,d}$ (UQ1.15×INT8); $y_d = \mathrm{sat}_8((A_d R + 2^{31}) \gg 32)$	$16(t+1)+1$

Skorlar (11×32 bit) ve üstel ağırlıklar (11×16 bit) birimin içindeki yazmaçlarda tutulur. AV aşamasında dış döngü d, iç döngü j üzerindedir; böylece her y_d, j döngüsü biter bitmez yazılabilir. Bir katmanın dikkat süresi yaklaşık 3 $(33\,(t+1)+53)$ çevrimdir; $t = 0$'da 258, $t = 10$'da 1 248 çevrim.

22 Bellek organizasyonu ve FPGA kaynak eşlemesi

Tasarımın bellek gereksinimi yaklaşık 95 KB ROM ve 4 KB RAM'dir. Büyük ROM'lar blok RAM'e (BRAM), küçük tamponlar ve tablolar dağıtık RAM'e (LUTRAM) veya yazmaçlara eşlenir. Aşağıdaki ayrıntılı eşlemeler hesapla elde edilmiş tahminlerdir. Bölümün sonunda açık kaynak Yosys sentezinin ölçtüğü sayılar verilmiştir (Tablo 20); kesin değerler ve zamanlama için Vivado gerekir.

Tablo 18. Bellekler ve beklenen gerçeklemeleri.

Bellek	Boyut	Okuma	Beklenen gerçekleme
Ağırlık ROM'u	10 416 × 64 bit (83,3 KB)	senkron	≈ 21 RAMB36 (512 × 72 düzeni)
Parametre ROM'u	1 304 × 64 bit (10,4 KB)	senkron	≈ 3 RAMB36
`wte` / `wpe`	144 / 528 × 16 bit	senkron	LUT-ROM veya RAMB18
GELU tablosu	768 × 8 bit	asenkron	LUT-ROM
exp tablosu	256 × 16 bit	asenkron	LUT-ROM
K ve V önbelleği	2 × 1 584 × 8 bit	asenkron	LUTRAM
Artık tampon	48 × 16 bit	asenkron, 3 port	yazmaç
xa, xb, qbuf	48, 192, 48 × 8 bit	asenkron	LUTRAM

Asenkron okumalı küçük tamponlar. LUTRAM, okuma adresi verildiği çevrimde veriyi kombinasyonel olarak verir. Bu sayede GEMV, LN ve dikkat birimlerinin durum makineleri adres-veri hizalaması için ek boru hattı aşaması gerektirmez. Bunun bedeli, okuma yolunun saat periyoduna dahil olmasıdır.

Tek yazma portu. Artık ve xb tamponlarına iki farklı kaynak yazar (gömme ile artık toplama; dikkat çıktısı ile GELU). Bu kaynaklar zaman içinde çakışmaz. RTL'de her tampon için tek bir yazma portu kombinasyonel olarak seçilir; sentez aracı tamponu tek portlu RAM olarak çıkarabilir.

Çarpıcı envanteri. DSP48E1 bloğu 25×18 bitlik işaretli bir çarpıcı, 48 bitlik bir toplayıcı ve boru hattı yazmaçları içerir; bu boyutu aşan çarpımlar birden fazla DSP'ye bölünür.

Tablo 19. Tasarımdaki çarpıcılar ve tahmini DSP48E1 kullanımı.

Yer	İşlem	Operandlar	Adet	Tahmini DSP
GEMV şeritleri	$W \cdot x$	8×8	8	0–8
Requant	$(acc + b) \cdot m_0$	33×17	1	2
LayerNorm	$d \cdot d$	32×32	1	3–4
LayerNorm	$d \cdot inv$	32×49	1	4–6
Dikkat	$q \cdot k$	8×8	1	0–1
Dikkat	$e \cdot v$	17×8	1	1
Dikkat	$(m - s) \cdot C_{exp}$	32×16	1	2
Dikkat	$A \cdot R$	32×49	1	4–6

Toplam tahmin 16–30 DSP'dir; Zynq-7020'de 220 DSP bulunur. Geniş çarpıcıların operand genişlikleri bilinçli olarak güvenli tutulmuştur. Bölüm 18'teki sınırlar gerçek ihtiyacı gösterir (d için 23 bit, inv için yaklaşık 27 bit, R için 18 bit). Genişlikler daraltılırsa DSP sayısı ve kritik yol kısalır. Hedef cihaz xc7z020'de 53 200 LUT, 106 400 FF, 140 RAMB36 ve 220 DSP bulunur. Tasarımın yaklaşık 24 RAMB36'lık ihtiyacı, cihaz BRAM'inin %17'si kadardır.

Açık kaynak sentez sonucu. Vivado bu geliştirme bilgisayarında (macOS, Apple Silicon) çalışmadığı için tasarım, açık kaynak **Yosys** 0.69 ile Xilinx 7-serisi hedefine bağlam dışı (out-of-context) sentezlenmiştir. SystemVerilog kodu önce `sv2v` ile Verilog'a çevrilir, ardından `synth_xilinx -family xc7` uygulanır (`make synth`, yaklaşık 26 s). Sonuçlar Tablo 20'tedir.

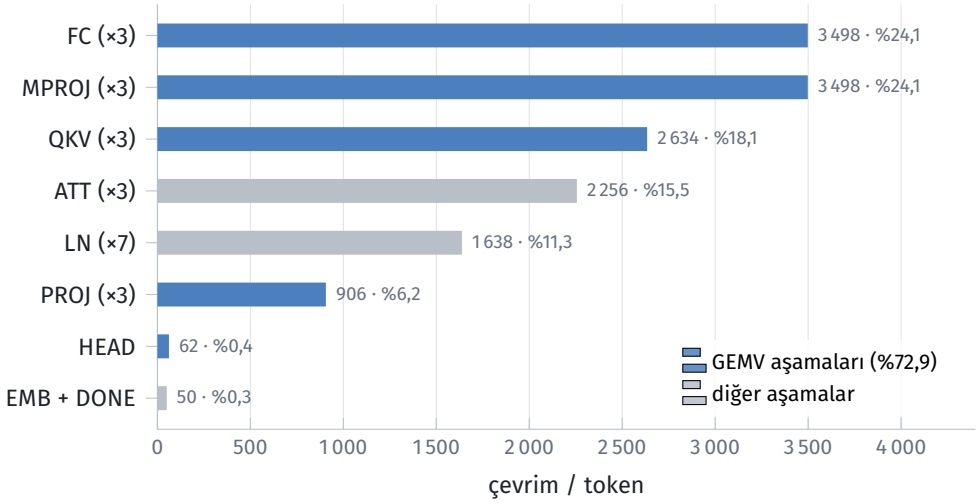
Tablo 20. `gpt_top`'un Yosys sentez sonucu (Xilinx 7-serisi, zamanlama analizi yok).

Kaynak	Kullanım	xc7z020 payı
LUT (mantık)	3 187	
LUT (dağıtık RAM ve SRL)	768	
LUT toplam	**3 955**	**%7,4**
Flip-flop	1 567	%1,5
BRAM (20 RAMB36 + 6 RAMB18)	23 RAMB36 eşd.	%16,4
DSP48E1	31	%14,1

BRAM sayısı, bellek haritasından yapılan tahminle (yaklaşık 24) uyuşur. DSP sayısı (31), tahmin aralığının (16–30) biraz üzerindedir; geniş çarpıcılar bildirilen genişlikleriyle sentezlenmiştir ve bit genişliği daraltması bu sayıyı düşürecektir. Yosys zamanlama analizi yapmaz; ulaşılabilir saat frekansı (Fmax) ve Vivado'nun kesin sayıları için Vivado sentezi gerekir. Vivado'nun sonuçları, özellikle LUT sayısında, Yosys'ten farklı olabilir.

23 Performans analizi

Bir token ortalama 14 544 çevrimde işlenir. Bu, 100 MHz'de 145 µs/token, yani saniyede yaklaşık 6 900 token demektir. Sürenin %72,9'u GEMV aşamalarında geçer (Şekil 19).



Şekil 19. Token başına çevrim profili (tüm katmanlar toplamı; Verilator, 729 dizi × 11 adım ortalaması). Testbench el sıkışmasıyla birlikte toplam 14 544 çevrimdir.

MLP'nin iki katmanı (FC ve MPROJ) tek başına sürenin yaklaşık yarısını alır; bu oran Bölüm 8'deki parametre dağılımıyla ($8C^2/12C^2$) uyumludur. Dikkat (%15,5) ve LayerNorm (%11,3) ardışık birimler oldukları için işlem paylarından (%4'ün altında) daha fazla süre alır.

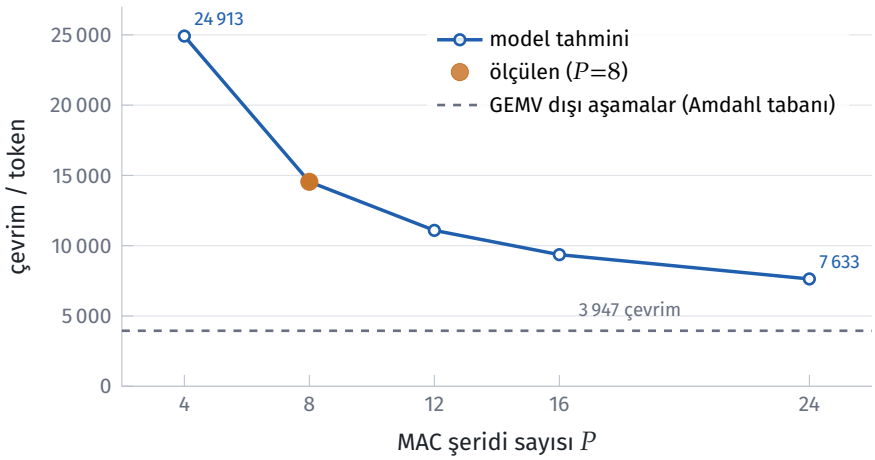
Analitik çevrim modeli. Bölüm 20 ve 21'daki çevrim sayıları birleştirilince, t konumundaki bir tokenın süresi kapalı biçimde yazılabilir:

$$\begin{aligned} T_{token}(t) &= \underbrace{49}_{\text{EMB}} + \underbrace{7 \cdot 234}_{\text{LN}} + \underbrace{3\,(878 + 302 + 1166 + 1166) + 62}_{\text{GEMV} = 10\,598} + \underbrace{9\,(33\,(t+1) + 53)}_{\text{dikkat}} + 1 \\ &\approx 12\,763 + 297\,(t+1). \end{aligned} \tag{44}$$

Model $t = 0$ için 13 060, $t = 10$ için 16 030 çevrim verir. Ortalaması 14 545 çevrimdir ve ölçülen 14 544 değeriyle bir çevrim farkla uyuşur.

Darboğaz ve Amdahl sınırı. Decode sırasında her ağırlık token başına bir kez kullanılır; aritmetik yoğunluk ağırlık baytı başına 1 MAC'tir. GEMV hızı bu yüzden çevrim başına okunabilen ağırlık baytıyla sınırlıdır: $P = 8$ şerit ROM'dan çevrim başına 8 bayt okur. GEMV dışındaki aşamalar 3 944 çevrim sürer. GEMV sonsuz hızlı olsa bile token süresi bu değerin altına inemez; yalnızca P artırılarak elde edilebilecek en büyük hızlanma $14\,542/3\,944 \approx 3{,}7\times$'tir.

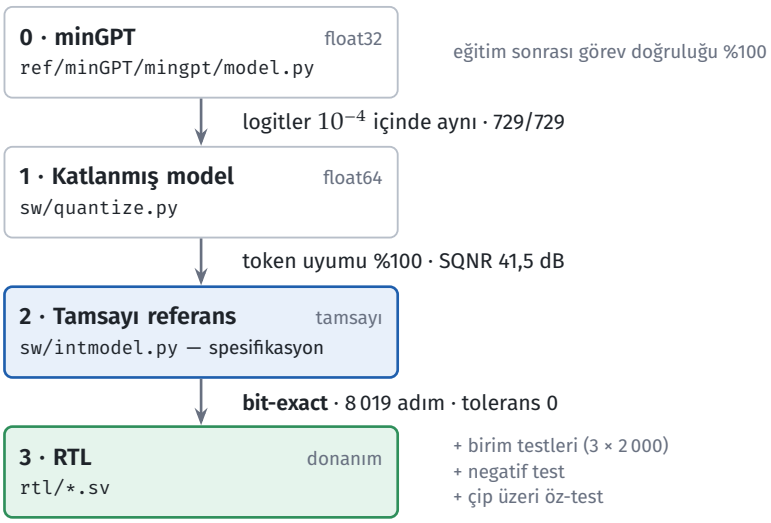
Paralellik derecesinin etkisi. P, bütün n_{out} değerlerini (144, 48, 192) bölmeli ve $n_{in} > P$ sağlanmalıdır; geçerli değerler 4, 8, 12, 16 ve 24'tür ($P = 32$ geçersizdir, çünkü 144 ve 48, 32'ye bölünmez). Şekil 20, analitik modelin tahminlerini gösterir: P'yi 8'den 24'e çıkarmak süreyi yalnızca 1,9 kat kısaltır. Daha fazla hızlanma için LayerNorm ve dikkat birimlerinin de paralelleştirilmesi gerekir.



Şekil 20. MAC şeridi sayısı P'nin token süresine etkisi (analitik model). $P = 8$ ölçülmüştür. Kesikli çizgi, GEMV dışındaki aşamaların oluşturduğu Amdahl tabanıdır.

24 Doğrulama metodolojisi

Donanımın doğruluğu, her biri bir öncekine karşı kontrol edilen dört referans seviyesiyle gösterilmiştir (Şekil 21). Son seviyede RTL, tamsayı referans modelle 8 019 decode adımının tamamında bit düzeyinde aynıdır.



Şekil 21. Referans model hiyerarşisi. Seviye 2, donanımın bit düzeyindeki spesifikasyonudur; seviye 2 ile 3 arasındaki karşılaştırma toleranssızdır.

Python tamsayıları sınırsız hassasiyetlidir. Bu yüzden RTL'deki her taşma, yanlış bit genişliği veya farklı bir yuvarlama, ilk etkilediği değerde bir uyumsuzluk olarak görünür.

Birim testleri (`tb/tb_arith.sv`). Aritmetik bloklar referans modele ihtiyaç duymayan matematiksel koşullarla test edilir: `div_seq` için $q\,d \le n < (q+1)\,d$, `isqrt_seq` için $r^2 \le n < (r+1)^2$ ve `requant` için davranışsal referansa eşitlik. Her blok 2 000 rastgele girdiyle sınanmıştır.

Sistem testi (`tb/tb_gpt_top.sv`). `sw/export.py`, tamsayı modelin 729 girdinin her biri için yaptığı 11 decode adımını `mem/tv.txt` dosyasına yazar. Testbench:

1. **kapalı döngü** çalışır: ilk 6 adımda girdi tokenları verilir, sonraki adımlarda DUT'nin kendi ürettiği token geri beslenir;
2. her adımda token, bütün logitler ve 48 artık değer referansla birebir karşılaştırılır;
3. dizi sonunda üretilen 6 tokenın girdinin sıralı hâli olup olmadığını ayrıca kontrol eder;
4. FSM durumlarına göre çevrim profilini ölçer (Şekil 19).

Tablo 21. Sistem testi sonuçları (Verilator 5.052).

Ölçüt	Değer
Kontrol edilen decode adımı	8 019
Bit-exact diziler	729 / 729
Doğru sıralanan diziler	729 / 729
Uyumsuz değer	0

Testin testi. Her zaman PASS veren bir testbench hiçbir şeyi kanıtlamaz. Bu yüzden altın dosyaya bilerek iki hata eklenmiştir: bir artık değere +1 ve değiştirilmiş bir token. Testbench ikisini de doğru dizi ve adımla raporlamış ve FAIL vermiştir.

> **Karşılaşılan bir SystemVerilog tuzağı.** Bir `initial` bloğunun içindeki döngüde ilk değerle tanımlanan değişkenler (`int x = $urandom();`) varsayılan olarak *static* ömürlüdür ve ilk değer yalnızca bir kez atanır (IEEE 1800, §6.21). Bu hatayla 2 000 rastgele test aslında aynı girdiyi 2 000 kez denerdi. Verilator'ın `IMPLICITSTATIC` uyarısıyla fark edilmiş ve değişkenler `automatic` yapılarak düzeltilmiştir.

Çip üzeri öz-test (`tb/tb_selftest.sv`). Bölüm 25'deki öz-test modülü simülasyonda iki kez çalıştırılmıştır: normal modda 729/729 dizi geçmiş, hata enjeksiyonu açıkken 729/729 dizi kalmıştır. Bir çalıştırma 116 637 813 çevrim sürer.

25 PYNQ-Z2 entegrasyonu

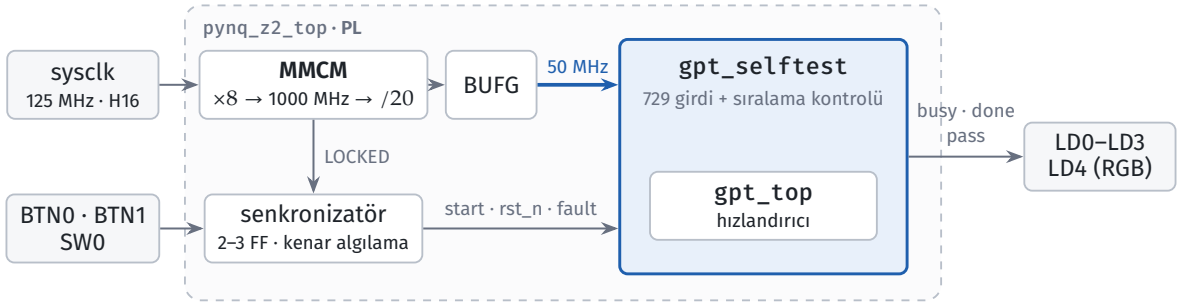
Hedef kart, Zynq-7020 (xc7z020clg400-1) içeren TUL PYNQ-Z2'dir. İlk kart hedefi, PC veya yazılım gerektirmeyen bir **çip üzeri öz-testtir**: FPGA 729 sıralama probleminin hepsini kendisi üretir, çözer ve kontrol eder; sonucu LED ile gösterir.

Tablo 22. Aday kartların karşılaştırması.

Kart	FPGA	RAMB36	DSP	Değerlendirme
PYNQ-Z2	Zynq-7020	140	220	**seçildi**: 2×Cortex-A9 + PYNQ (Python)
Basys 3	Artix-7 35T	50	90	sığar, ancak büyümeye yer az; işlemci yok
Zybo Z7-20	Zynq-7020	140	220	aynı çip; resmî PYNQ desteği yok
Kria KV260	Zynq US+ K26	144 (+64 URAM)	1 248	bu boyut için gereğinden büyük

PYNQ-Z2'nin seçilme nedenleri şunlardır: BRAM'in yalnızca yaklaşık %17'sinin kullanılması, ücretsiz Vivado ML Standard sürümüyle desteklenmesi ve kart üzerindeki ARM işlemci. Bu işlemci, sonraki aşamada hızlandırıcıyı Python'dan sürmeye ve aynı karttaki CPU ile karşılaştırmaya imkân verir.

Öz-test modülü (`rtl/gpt_selftest.sv`). Modül, girdileri tabanı 3 olan 6 basamaklı bir sayaçla (odometre) üretir. Her girdi için hızlandırıcıyla 11 adımlık kapalı döngü üretim yapar. Üretilen 6 token iki koşulla kontrol edilir: *monotonluk* (her token bir öncekinden küçük olmamalı) ve *histogram eşitliği* (her rakamın çıktıdaki sayısı girdidekine eşit olmalı). Bu iki koşul birlikte "çıktı, girdinin sıralı hâlidir" önermesine denktir; çipte altın veri saklanmaz. Hata enjeksiyonu açıkken her dizinin ilk çıktısı farklı geçerli bir tokenla değiştirilir. Histogram bu durumda kesinlikle değişeceği için her dizi kalmalıdır.



Şekil 22. PYNQ-Z2 üst modülü. 125 MHz kart saatinden MMCM ile 50 MHz üretilir. Butonlar senkronize edilir, öz-test sonucu LED'lerde gösterilir.

Saat ve reset. Kart, PL'ye H16 pininden 125 MHz saat verir. Bu saatten bir MMCM ile tasarım saati üretilir:

$$f_{VCO} = 125\ \text{MHz} \times 8 = 1000\ \text{MHz}, \qquad f_{clk} = f_{VCO}/20 = 50\ \text{MHz}. \tag{45}$$

VCO frekansı, -1 hız sınıfı 7-serisi MMCM'nin 600–1200 MHz aralığındadır. 50 MHz, zamanlama raporu görülene kadar bilinçli olarak temkinli seçilmiştir; rapor uygunsa bölen 10'a düşürülerek 100 MHz'e çıkılabilir. PL'ye bağlı bir reset pini yoktur; reset MMCM'nin `LOCKED` sinyali ve BTN1 ile üretilir.

Tablo 23. Kart arayüzü.

Öğe	Anlamı
LD0	yanıp söner: bitstream çalışıyor
LD1 / LD2	test sürüyor / bitti
LD3	hata enjeksiyonu açık (SW0)
LD4 (RGB)	mavi: sürüyor · yeşil: 729/729 doğru · kırmızı: hata var
BTN0 / BTN1 / SW0	testi yeniden başlat / reset / hata enjeksiyonu

Bir çalıştırma 116,6 milyon çevrim sürer: 50 MHz'de yaklaşık 2,3 s, 100 MHz'de 1,17 s. Kart akışı iki komuttan oluşur: `scripts/vivado_build_pynq_z2.tcl` sentez, yerleştirme, yönlendirme ve bitstream üretimini yapar ve kaynak, zamanlama, güç raporlarını yazar; `scripts/vivado_program.tcl` kartı JTAG üzerinden programlar. Beklenen sonuç: LD4 yaklaşık 2,3 s mavi yanar, sonra yeşil olur. Bu adımlar hazırdır, ancak raporun tarihinde henüz çalıştırılmamıştır.

26 Sınırlamalar ve gelecek çalışmalar

Mevcut tasarım, işlevsel doğruluğu kanıtlanmış bir temel sürümdür. Bir sonraki adım onu kart üzerinde çalıştırmak, ardından ölçerek iyileştirmektir.

Sınırlamalar.

- **Fiziksel doğrulama yok:** Vivado sentezi, zamanlama kapanışı ve kart üzerinde çalıştırma henüz yapılmamıştır; kaynak kullanımı yalnızca Yosys ile ölçülmüştür. 100 MHz bir hedeftir, ölçülmüş değildir.
- **Model boyutu:** model küçüktür (85 728 parametre) ve dar bir görevde eğitilmiştir; metin üreten bir dil modeli değildir.
- **Ölçek:** bütün ağırlıkların BRAM'e sığması varsayılmıştır. Bu varsayım Zynq-7020'de yaklaşık 600 bin INT8 parametreye kadar geçerlidir.
- **Kod çözme:** yalnızca açgözlü kod çözme vardır; sıcaklıklı örnekleme ve top-k yoktur.
- **Bit genişlikleri:** geniş çarpıcıların genişlikleri güvenli tarafta tutulmuştur; kritik yol ve DSP sayısı henüz optimize edilmemiştir.

Gelecek çalışmalar.

1. **Kartta çalıştırma ve ölçüm:** PYNQ-Z2 bitstream'i, kaynak/Fmax/güç raporları ve yeşil LED.
2. **Paralellik taraması:** $P \in \{4, 8, 12, 16, 24\}$ için çevrim, BRAM, DSP ve Fmax değişimi (Şekil 20). P, `quantize.py` içinde tek bir parametredir.
3. **Tek geçişli LayerNorm:** $\sum d^2 = N(N\sum x^2 - s^2)$ özdeşliğiyle LN başına 48 çevrim kazancı. (Kısım IV'teki motorda gerçeklenmiştir.)
4. **Paralel dikkat:** SCORE ve AV aşamalarında $d_h = 16$ boyutu üzerinden paralel MAC; dikkat süresinde yaklaşık 16 kat azalma. (Kısım IV'teki motorda $d_h = 8$ şeritle gerçeklenmiştir.)
5. **Bit genişliği analizi:** Bölüm 18'teki sınırlardan türetilen dar operandlar ve boru hattı bölme ile daha yüksek Fmax.
6. **PS entegrasyonu:** AXI4-Lite sarmalayıcı, Vivado blok tasarımı ve PYNQ Python sürücüsü. Böylece aynı karttaki ARM işlemciyle (NumPy/PyTorch) gecikme ve enerji karşılaştırması yapılabilir.
7. **Daha büyük görev:** minGPT'nin `projects/chargpt` örneğiyle karakter düzeyinde metin üretimi. Mimari aynı kalır; büyük modellerde ağırlıklar DDR'dan akış hâlinde okunur.

KISIM IV

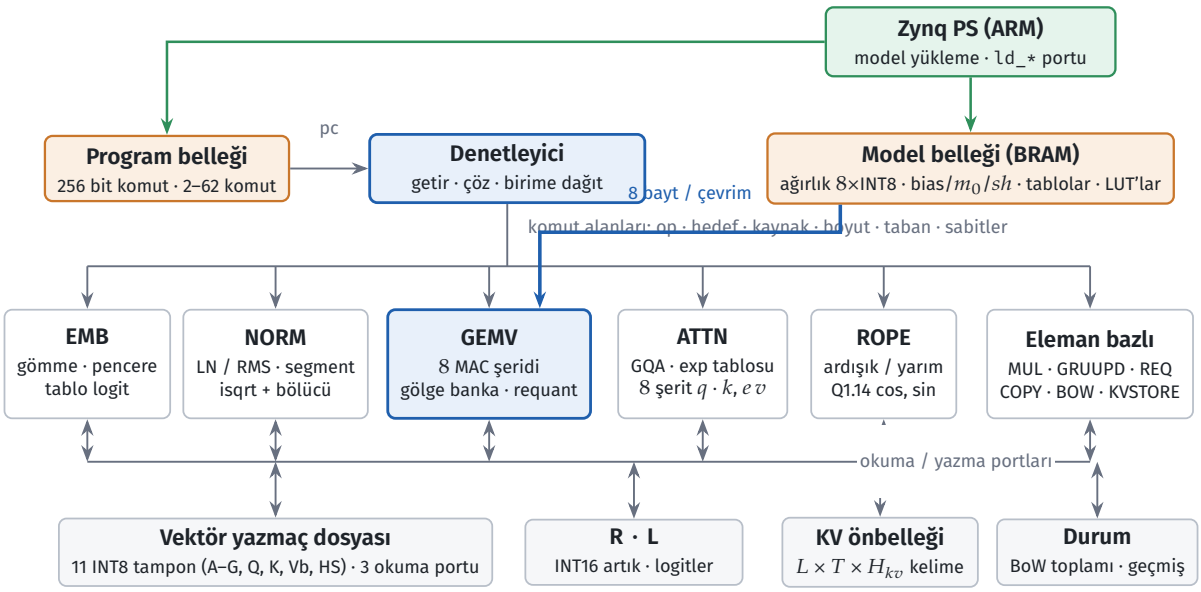
Model Ailesi Donanımda: Mikrokodlu Motor ve Karşılaştırma

27 Mikrokodlu motor: tek RTL, dokuz mimari

Kısım III'teki hızlandırıcının kontrol FSM'i minGPT'nin işlem sırasını sabit olarak kodlar. Ailenin dokuz mimarisini aynı donanımla çalıştırmak için bu FSM, küçük bir *mikrokod* yorumlayıcısıyla değiştirilmiştir (`rtl/zoo/zoo_core.sv`). Python derleyicisi her modeli token başına çalışan düz bir komut listesine çevirir. Denetleyici bu listeyi program belleğinden okur, her komutu ilgili birime dağıtır ve birim bitince bir sonrakine geçer (Şekil 23). Modele özgü her şey (program, ağırlıklar, yeniden kuantizasyon sabitleri, gömme ve RoPE tabloları, aktivasyon tabloları) belleklerde durur. RTL kaynağı bütün modeller için aynıdır.

Komut kümesi. Her komut 256 bittir. Komutta işlem kodu, hedef ve iki kaynak tampon, bir mod alanı, dört adet 16 bitlik boyut alanı, iki bellek taban adresi ve iki sabit (64 ve 32 bit) bulunur. Sabitler; yeniden kuantizasyon çarpanı (m_0, sh), NORM'un K ve E sabitleri ya da softmax'ın üs ölçeği gibi derleme anında bilinen değerleri taşır. Tablo 24, on beş işlemi ve onları kullanan modelleri listeler. Her işlemin tamsayı semantiği `sw/zoo/quant.py` içindeki `IntRunner` tarafından tanımlanır. RTL bu tanımla bit düzeyinde eşleşmek zorundadır.

GEMV birimi. minGPT tasarımındaki çıkış-durgun motor (Bölüm 20) olduğu gibi kullanılır: 8 MAC şeridi, INT32 akümülatörler, gölge banka ve seri yeniden kuantizasyon birimi. Bir bloğun yeniden kuantizasyonu, sonraki bloğun birikimiyle örtüşür. İki genelleme eklenmiştir. (i) Giriş vektörü iki tamponun birleşimi olabilir; RNN ve GRU hücrelerindeki $W[x_t; h_{t-1}]$ çarpımı tek komutla yapılır. (ii) Çıkış sayısı 8'in katına tamamlanır, fazla çıkışların yazımı maskelenir. Ağırlık belleği her çevrimde 8 bayt verir. Bir $n_{\text{in}} \times n_{\text{out}}$ katman $n_{\text{in}}\lceil n_{\text{out}}/8 \rceil$ çevrim sürer.



Şekil 23. Mikrokodlu motorun blok şeması. Denetleyici 256 bitlik komutları çözüp yedi birimden birine verir. Birimler ortak bir vektör yazmaç dosyasını, INT16 artık/logit belleklerini, KV önbelleğini ve durum belleklerini paylaşır. Model belleği, Zynq'in ARM çekirdeği tarafından çalışma anında doldurulabilir (LOADABLE).

NORM birimi. minGPT'deki üç geçişli LayerNorm, Bölüm 26'de önerilen özdeşlikle tek geçişe indirilmiştir:

$$Q = \sum_{i=1}^{n} (nx_i - s)^2 + E = n\Big(n \sum_i x_i^2 - s^2\Big) + E, \qquad s = \sum_i x_i. \tag{46}$$

Böylece $\sum x_i$ ve $\sum x_i^2$ aynı geçişte toplanır. RMSNorm'da $Q = \sum x_i^2 + E$'dir. Karekökten önce $Q, k = \max(0, \lfloor (40 - \text{bitlen}(Q))/2 \rfloor)$ kadar kaydırılarak 39–40 bite normalize edilir. Ardından $\text{inv} = \lfloor K 2^k / \lfloor \sqrt{Q\,4^k} \rfloor \rfloor$ hesaplanır. Bu normalizasyon, Q küçük olduğunda karekökün göreli hassasiyetini sabit tutar. INT8 çıkışlı QK-norm gibi küçük segmentlerde bu önemlidir. Segment başına maliyet $2n$ çevrim artı 28 çevrimlik karekök ve bölmedir.

Sınırlı bölümlü bölücü. Bölme sonucunun 2^{Q_B}'den küçük olduğu biliniyorsa, W bitlik bir restoring bölmenin ilk $W - Q_B$ bölüm biti sıfırdır. Bu adımlardan sonraki kısmi kalan da tam olarak $\lfloor \text{num}/2^{Q_B} \rfloor$'dur. `zdiv.sv` bölmeye doğrudan bu noktadan başlar ve yalnızca Q_B çevrim harcar. NORM'da $\text{inv} < 2^{40}$ olduğu için 64 yerine 41 çevrim yeter. Softmax'ın $2^{32}/E$ karşılığında $E \geq 2^{15}$ olduğu için 34 yerine 18 çevrim yeter. Bu değişiklik GPT-2'de token başına çevrimi %5, nanochat'te %9 azaltmıştır.

Dikkat ve RoPE. Bütün transformerlerde baş boyutu $d_h = 8 = P$'dir. KV önbelleğinin her kelimesi bir başın bir konumdaki 8 baytını tutar. Böylece $q \cdot k_j$ skoru ve $e_j v_j$ birikimi tek çevrimde 8 şeritle hesaplanır. Bir baş $2(t+1)$ çevrim, 18 çevrimlik bölme ve 8 çıkış çevrimi sürer. GQA'da H/H_{kv} ardışık sorgu başı aynı KV kelimelerini okur. Softmax, minGPT'deki tamsayı yöntemin aynısıdır (Bölüm 18): $2^{-k/256}$ tablosu ve tek karşılık. RoPE birimi bir başın kelimesini okur, cos ve sin değerlerini Q1.14 tablolardan alır, çiftleri ardışık (llama2.c) ya da yarım bölmeli (nanochat) düzende döndürüp kelimeyi geri yazar.

Tablo 24. Mikrokod işlemleri. R: INT16 artık akış, L: INT16 logitler, h: tekrarlayan durum (Q0.7).

İşlem	Anlamı	Kullanan modeller
`EMB16`	$R \leftarrow \text{wte}[x_t]$ $(+\,\text{wpe}[t])$, INT16 doyurmalı	BoW, transformerler
`EMB8 / EMBWIN`	INT8 gömme / son k tokenın gömmelerinin birleşimi	RNN, GRU / MLP
`TABLOGIT`	$L \leftarrow T[x_t]$ (logit tablosu)	Bigram
`HRESET`	$t = 0$ iken $h \leftarrow h_0$	RNN, GRU
`NORM`	LayerNorm veya RMSNorm, n uzunluklu segmentler; INT16 ya da INT8 kaynak/hedef	transformerler
`GEMV`	$y = W[x;\, x'] + b$; mod: INT8 çıkış, artığa ekleme, tablo aktivasyonu, logit	Bigram hariç hepsi
`ROPE`	baş başına döndürme, ardışık ya da yarım bölmeli çiftler	Llama 2, nanochat
`KVSTORE / ATTN`	k_t, v_t'yi önbelleğe yaz / GQA'lı tamsayı softmax ve ağırlıklı ortalama	transformerler
`MUL`	$z = \text{rq}(a \odot b)$	GRU, Llama 2
`GRUUPD`	$h \leftarrow h + \lfloor (z\,(\tilde{h} - h) + 64)/2^7 \rfloor$	GRU
`COPY, REQ`	tampon kopyalama; INT16 → INT8 yeniden kuantizasyon	RNN; BoW, microgpt
`BOW`	$S \mathrel{+}= R,\;\; R \mathrel{+}= \lfloor S \cdot \lfloor 2^{20}/(t+1) \rceil / 2^{20} \rceil$	BoW

Özelleştirme ve yüklenebilir bellekler. İki sentez parametresi vardır. `SPECIALIZE=1` olduğunda, derleyicinin ürettiği `OPS_USED` maskesinde bulunmayan birimler sabit koşullarla devre dışı kalır ve sentezde silinir. `LOADABLE=1` olduğunda ise model bellekleri ROM yerine bir yazma portlu RAM olur. Bu port Zynq'in işlemci tarafından (PS) sürülür; tek bitstream bütün modelleri çalıştırabilir. Bu ayrımın ölçüm açısından önemli bir sonucu vardır. Bellekler sabit ROM olarak bırakıldığında Yosys programı da sabit kabul eder ve kullanılmayan bütün mantığı katlar. Örneğin Bigram için "genel" motor 82 LUT'a inmiştir. Bu yüzden bu bölümdeki bütün kaynak sayıları `LOADABLE=1` ile alınmıştır.

28 Derleyici ve doğrulama zinciri

minGPT için kurulan doğrulama zinciri (Bölüm 17–18) her model için otomatik olarak çalıştırılmıştır. Zincirin halkaları şunlardır:

1. **PyTorch → IR.** `ir.py` her modeli token başına çalışan bir programa derler. Bu adımda doğrusal olmayan katmanların afin kısımları ve dikkat ölçeği ağırlıklara katlanır. Float64 IR yorumlayıcısının logitleri, PyTorch modelininkilerle her modelde en fazla $2 \cdot 10^{-5}$ farklıdır (stories260K'da $9 \cdot 10^{-5}$).

2. **IR → tamsayı program.** Kalibrasyon kümesi üzerinde aralıklar ölçülür (Bölüm 29). `quant.py` ağırlıkları satır başına INT8'e çevirir; giriş ölçeklerini ağırlıklara katlar ve çıkış kanalı başına (m_0, sh) üretir. `IntRunner` bu programı yalnızca tamsayılarla çalıştırır ve donanımın spesifikasyonu olarak kullanılır.

3. **Tamsayı program → bellekler.** `export_hw.py` programı 256 bitlik komutlara, ağırlıkları 64 bitlik kelimelere, sabitleri ve tabloları ayrı belleklere yazar. Model başına bir `zoo_pkg.sv` paketi de üretir. Ayrıca `IntRunner`'dan altın test vektörleri çıkarır.

4. **RTL.** `tb_zoo.sv`, Verilator'da her adımın *bütün* logitlerini altın vektörlerle karşılaştırır. Her isim modeli için 50 test ismi (354 adım × 27 logit) hem genel hem özelleşmiş motorda çalıştırılmıştır. stories260K için iki test hikâyesi (1 024 adım × 512 logit) kullanılmıştır. Ek olarak motor, kendi ürettiği tokenı geri besleyerek 200 tokenlık bir hikâye üretmiştir.

Sonuç On modelin (dokuz isim modeli ve stories260K) hepsinde RTL, tamsayı referansla *bit düzeyinde* aynıdır: hiçbir adımda tek bir logit bile farklı değildir. Kapalı döngü üretimde 200 tokenın 200 tanesi referansla aynıdır.

29 Nicemleme: kalibrasyon ve hata kaynakları

minGPT'de aktivasyon ölçekleri kalibrasyon kümesindeki en büyük mutlak değerden seçilmişti. Normalize edilmiş vektörlerin ölçeği ise ispatlanmış sınırdan geliyordu ($|\hat{x}_i| \le \sqrt{n}$; Tablo 9). Bu yöntem isim modellerinde iyi çalışmıştır. stories260K'da ise INT8 kaybı +0,052 nat/token'a çıkmıştır. Kaybın kaynağını bulmak için float IR'da tek seferde bir tensör türü, tamsayı programın kullandığı ızgaraya yuvarlanmıştır (`ablate.py`). Sonuçlar Tablo 25'dadır.

Tablo 25. Sahte nicemleme ablasyonu: yalnızca belirtilen tensör türü nicemlendiğinde test kaybındaki artış (nat/token; maksimum kalibrasyon, sınırdan NORM ölçeği; isim modelleri için 200 test ismi, stories260K için 20 test hikâyesi). Ağırlıkların payı ihmal edilebilir; kaybın neredeyse tamamı aktivasyonlardan gelir. "–": modelde bu işlem yok.

Nicemlenen tensör	GPT-2	Llama 2	stories260K
ağırlıklar (satır başına INT8)	+0,0006	−0,0010	+0,0011
q, k, v ve doğrusal çıkışlar (INT8)	+0,0002	−0,0004	+0,0094
aktivasyon tablosu giriş/çıkışı	+0,0028	+0,0035	+0,0054
eleman bazlı çarpım (SwiGLU)	–	+0,0089	+0,0128
normalize edilmiş vektörler	+0,0068	+0,0037	+0,0096
artık akış (INT16)	−0,0001	+0,0001	−0,0000
logitler (INT16)	+0,0000	−0,0000	−0,0000
hepsi birlikte	+0,0077	+0,0115	+0,0436

Tablo iki şeyi gösterir. Birincisi, satır başına INT8 ağırlıkların maliyeti 10^{-3} düzeyindedir; INT16 artık akış ve logitlerin maliyeti ise sıfırdır. İkincisi, kayıp INT8 aktivasyonlara dağılmıştır: SwiGLU çarpımı, q, k, v çıkışları ve normalize edilmiş vektörler. Normalize vektörlerin sınırı $\sqrt{64} = 8$'dir ve ölçek bu sınırdan 2^{-3} seçilir. Oysa gerçek değerler bu sınıra nadiren yaklaşır; birim başına yalnızca 8 seviye kalır. Buna karşı iki standart yöntem denenmiştir: (i) aktivasyon ölçeğini en büyük değer yerine $|x|$'in yüksek bir yüzdeliğinden seçmek (iki geçişli kalibrasyon, 4 096 kutulu histogram); (ii) normalize çıkışların ölçeğini ölçülen aralıktan seçmek, ama sınırdan türetilen ölçekten kaba olmamak şartıyla. İkisi de donanımda değişiklik gerektirmez; değişen yalnızca (m_0, sh) ve kaydırma sabitleridir. Tamsayı programın kendisiyle ölçülen sonuçlar Tablo 26'dadır.

%99,99 yüzdelik ve kalibre NORM ölçeği birlikte stories260K'nın kuantizasyon kaybını yarıdan fazla azaltır. Bu yapılandırma *bütün* modeller için seçilmiştir (`quant.CALIB`); sonraki tablolar onunla üretilmiştir. Yöntemin tamamı aynı kaldığı için mimariler arası karşılaştırma adildir.

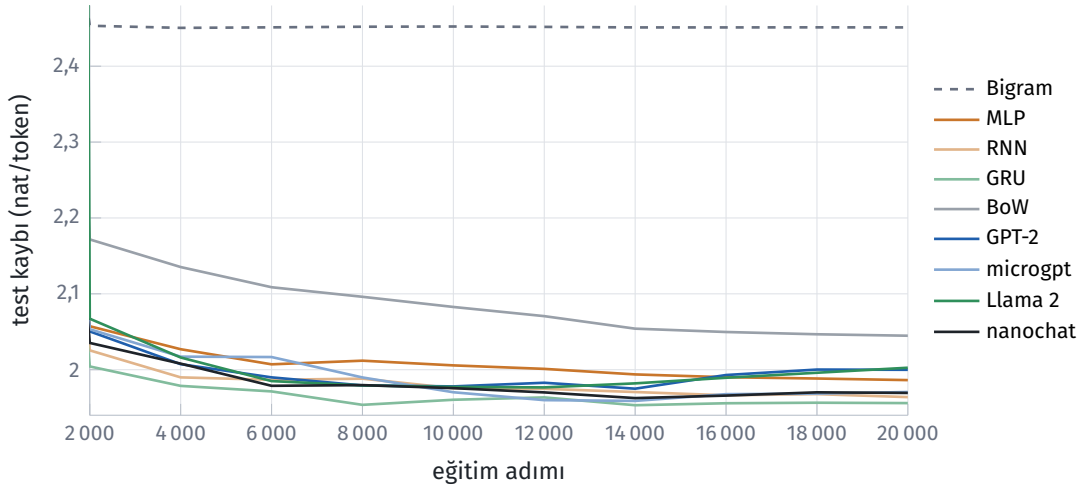
Tablo 26. Kalibrasyon yöntemleri, stories260K (bit-exact tamsayı program, 20 test hikâyesi). "Yüzdelik": INT8 aktivasyon ölçeğinin seçildiği $|x|$ yüzdeliği.

Yüzdelik	NORM ölçeği	INT8 kayıp	Δ	argmax uyumu (%)
maks.	sınır	1,4648	+0,0517	88,5
maks.	kalibre	1,4533	+0,0401	90,0
99,99	sınır	1,4474	+0,0343	90,8
99,9	kalibre	1,4407	+0,0276	91,3
99,99	kalibre	1,4360	+0,0229	91,9
99,999	kalibre	1,4474	+0,0343	91,1

Etkisi küçük modellerde karışıktır: GPT-2'nin kaybı azalmış, microgpt'ninki biraz artmıştır. Ama her durumda 10^{-2}'nin altında kalır.

30 Hat A: names.txt üzerinde dokuz mimari

Dokuz model makemore'un `names.txt` verisinde aynı koşullarla eğitilmiştir. Veri 32 033 isimdir; tohum 3407 ile karıştırılıp 31 033 eğitim ve 1 000 test ismine ayrılmıştır ($V = 27$, $T = 16$). Her model 20 000 adım eğitilmiştir: yığın 64, AdamW, öğrenme oranı $3 \cdot 10^{-3}$'ten kosinüsle 10^{-4}'e, 500 adım ısınma, ağırlık sönümü 0,01, gradyan kırpma 1,0. Bigram dışındaki bütün modeller yaklaşık 50 bin parametreye ayarlanmıştır (Tablo 27). Eğitim eğrileri Şekil 24'dadır.



Şekil 24. names.txt üzerinde test kaybı (nat/token, her 2 000 adımda bir ölçülmüştür). Bütün modeller aynı optimizasyon ayarlarıyla ve yaklaşık aynı parametre bütçesiyle eğitilmiştir.

Kalite. Aynı bütçede en iyi model GRU'dur (float 1,9560). Onu RNN (1,9639), nanochat (1,9694) ve microgpt (1,9706) izler. Bu ölçekte dikkat mekanizmasının üstünlüğü görünmez. İsimler kısadır ($T = 16$) ve 50 bin parametrelik bir RNN, bu kadar kısa bir geçmişi sabit boyutlu durumda kayıpsıza yakın özetleyebilir. Transformerler arasında nanochat (RMSNorm, ReLU^2, QK-norm) ve microgpt klasik GPT-2'den iyidir. Llama 2 ise bu bütçede GPT-2 ile aynı düzeydedir: SwiGLU'nun üç matrisi, aynı parametre sayısında daha dar bir gizli katman demektir. Nedensel ortalama (BoW) ise sırayı yalnızca konum gömmesiyle görebildiği için en

Tablo 27. Test kaybı (nat/token, 1 000 test ismi, ≈7 500 token). Δ: INT8 tamsayı programın kaybı eksi float kayıp. "argmax": iki modelin en olası sonraki karakter konusunda anlaştığı adımların oranı. Son satır Hat B'dir (50 TinyStories hikâyesi).

Model	Parametre	Float	INT8	Δ	argmax (%)
Bigram	729	2,4511	2,4511	−0,0000	100,0
MLP	50 459	1,9863	1,9874	+0,0011	98,1
RNN	49 347	1,9639	1,9671	+0,0032	96,1
GRU	48 823	1,9560	1,9563	+0,0003	98,3
BoW	49 979	2,0448	2,0445	−0,0003	97,0
GPT-2	53 120	1,9999	2,0016	+0,0017	96,3
microgpt	51 392	1,9706	1,9738	+0,0032	95,3
Llama 2	50 304	2,0026	2,0039	+0,0013	93,5
nanochat	46 784	1,9694	1,9750	+0,0055	93,7
stories260K	260 032	1,4003	1,4220	+0,0217	92,0

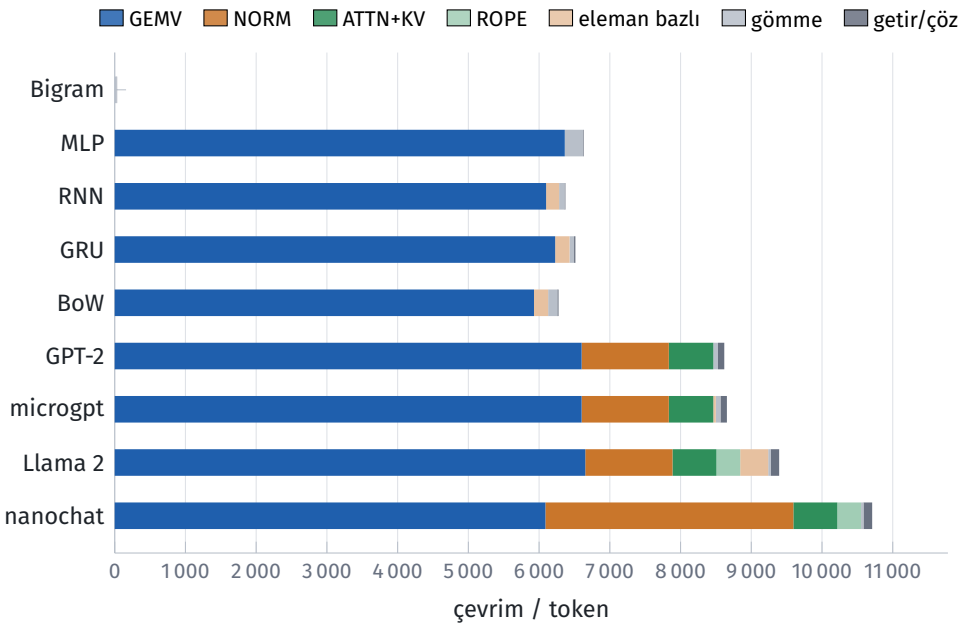
zayıf öğrenen modeldir. INT8 kaybı bütün modellerde 10^{-2}'nin altındadır; Bigram'da sıfırdır, çünkü tablo doğrudan INT16 logittir.

Performans. Tablo 28 her modelin donanım maliyetini verir. GEMV verimi, ideal çevrim sayısının ($n_{\text{in}}\lceil n_{\text{out}}/8\rceil$ toplamı) ölçülen GEMV çevrimlerine oranıdır. Yeniden kuantizasyon gölge bankada sonraki blokla örtüştüğü için verim %94,3 ile %99,6 arasındadır. Kayıp, her GEMV komutunun sonundaki birkaç çevrimlik boşaltmadan gelir; bu yüzden küçük katmanları çok olan transformerlerde biraz daha düşüktür. Parametre bütçesi eşit olduğu için GEMV süresi de yaklaşık eşittir (6 000–6 700 çevrim). Mimariler arasındaki farkı GEMV dışındaki işlemler belirler (Şekil 25). Transformerlerde NORM ve dikkat birlikte token süresinin %20–40'ını alır. Tekrarlayan modellerin ek yükü ise birkaç yüz çevrimdir.

Tablo 28. Token başına donanım maliyeti (Verilator, test dizilerinin ortalaması). "Komut": mikrokod programının uzunluğu. "Durum": çıkarım sırasında saklanan bayt; KV önbelleği konum başına büyür, diğerleri sabittir.

Model	Komut	GEMV MAC	Çevrim	µs @ 100 MHz	GEMV verimi (%)	Durum
Bigram	2	0	36	0,4	–	–
MLP	4	49 808	6 632	66,3	99,6	16 B
RNN	6	47 656	6 374	63,7	99,6	184 B
GRU	9	47 100	6 515	65,2	99,1	100 B
BoW	8	46 784	6 277	62,8	99,3	256 B
GPT-2	44	50 016	8 620	86,2	95,0	256 B/konum
microgpt	45	50 016	8 657	86,6	95,0	256 B/konum
Llama 2	60	50 016	9 396	94,0	94,3	128 B/konum
nanochat	61	45 920	10 711	107,1	94,6	128 B/konum
stories260K	74	259 328	59 092	590,9	98,6	320 B/konum

Kaynaklar. Tablo 29'te iki yapılandırma yan yana verilmiştir. Genel motor bütün birimleri içerir ve her modeli çalıştırabilir. Mantık maliyeti yaklaşık 5 900–8 900 LUT ve 60 DSP'dir; farklar çoğunlukla bellek boyutlarından kaynaklanır. Özelleşmiş motorda yalnızca modelin kul-



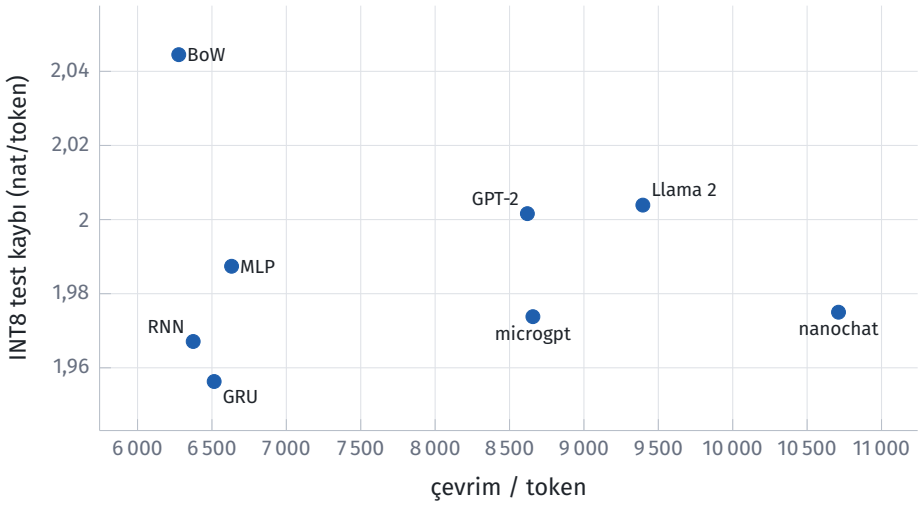
Şekil 25. Token başına çevrimlerin işlem türlerine dağılımı (50 test ismi ortalaması). Parametre bütçesi eşit olduğundan GEMV payı yaklaşık sabittir; mimariyi ayıran, normalizasyon ve dikkatin ek maliyetidir. nanochat'te QK-norm, katman başına altı ek NORM segmenti getirir.

landığı birimler kalır. Dikkat öncesi modellerin özelleşmiş motoru 2 100–4 300 LUT ve 14–19 DSP'ye iner. Transformerlerinki ise NORM ve dikkat birimleri yüzünden 50 civarında DSP tutar. DSP'lerin çoğu bu birimlerin geniş çarpıcılarındadır: NORM'un 26×42 bitlik ölçekleme çarpımı ve dikkat çıkışının 34×19 bitlik çarpımı gibi. BRAM kullanımı Bigram dışında 14–20 RAMB36'dır; ağırlıklar her modelde yaklaşık 50 KB'tır.

Kalite–hız ilişkisi. Şekil 26 iki ekseni birleştirir. Bu ölçekte Pareto sınırını Bigram (en hızlı, en kötü), BoW, RNN ve GRU oluşturur. GRU hem en düşük kayba sahiptir hem de transformerlerden %24–39 daha az çevrim harcar. Transformerlerin ek maliyeti bu görevde kaliteye dönüşmemektedir. Bu sonuç bağlam uzunluğuna bağlıdır. Tekrarlayan modelin durumu sabittir ve uzun bağlamlarda geçmişi sıkıştırmak zorundadır; dikkat ise geçmişi KV önbelleğinde saklar. Hat B bu yüzden uzun bir bağlamla ($T = 512$) ele alınmıştır.

Tablo 29. Yosys sentezi (Xilinx 7-serisi, xc7z020 hedefi; zamanlama yok). "Genel": bütün birimler; "özel": yalnızca modelin kullandığı birimler. İki yapılandırmada da model bellekleri yüklenebilir RAM'dir; BRAM sayısı iki yapılandırmada aynıdır.

	Genel motor			Özelleşmiş motor			
Model	**LUT**	**FF**	**DSP**	**LUT**	**FF**	**DSP**	**BRAM**
Bigram	5 855	2 530	60	1 529	1 137	14	0,5
MLP	8 094	2 439	60	2 502	1 049	14	14,5
RNN	7 195	2 441	60	2 161	1 051	14	14,0
GRU	7 164	2 436	60	3 239	1 128	16	15,5
BoW	8 917	2 420	62	4 247	1 143	19	16,0
GPT-2	7 206	2 273	60	5 850	2 140	49	19,5
microgpt	7 162	2 273	60	5 990	2 140	50	19,5
Llama 2	7 278	2 272	60	6 622	2 267	57	19,0
nanochat	6 954	2 270	60	6 179	2 265	56	18,0
stories260K	7 918	2 103	60	7 194	2 097	57	134,0



Şekil 26. INT8 test kaybı ile token başına çevrim (names.txt, $T = 16$). Sol alt köşe iyidir. Bigram (36 çevrim, 2,4511 nat/token) ölçeği bozmamak için gösterilmemiştir.

31 Hat B: stories260K ile hikâye üretimi

Karpathy'nin TinyStories serisinin en küçük modeli olan stories260K, llama2.c biçiminde hazır eğitilmiş olarak alınmıştır [31]. Yapılandırması şöyledir: 5 katman, $C = 64$, 8 sorgu ve 4 KV başı ($d_h = 8$), SwiGLU gizli boyutu 172, 512 tokenlık BPE sözlüğü, bağlam 512; toplam 260 032 parametre. Model, Hat A'daki `Llama` sınıfına doğrudan yüklenir. Bu sınıf llama2.c'nin `model.py`'sini birebir izler. Tokenizer, `run.c`'deki `encode` fonksiyonunun Python karşılığıdır. Ağırlıklar ve tokenizer `data/stories260K/` altındadır. Doğrulama için TinyStories doğrulama kümesinin ilk 400 hikâyesi kullanılmıştır: ilk 100'ü kalibrasyon, 200. hikâyeden sonrakiler test içindir.

Kalite. 50 test hikâyesinde (23 129 token) float kayıp 1,4003, INT8 kayıp 1,4220 nat/token'dır (Δ +0,0217); argmax uyumu %92,0'dir. Isim modellerine göre büyük olan bu fark, daha uzun bağlam ve daha geniş aktivasyon dağılımlarından kaynaklanır (Bölüm 29).

Donanım. Aynı RTL, yalnızca daha büyük belleklerle stories260K'yı çalıştırır. Bellekler 134,0 RAMB36 tutar; bu, xc7z020'nin 140 BRAM'inin neredeyse tamamıdır. Bunun yaklaşık 260 KB'ı INT8 ağırlıklar, 160 KB'ı ise 512 konumluk KV önbelleğidir ($2 \times 5 \times 512 \times 32$ bayt). Bağlamı 256'ya indirmek KV önbelleğini yarıya düşürür. Mantık tarafı isim modelleriyle aynı düzeydedir (genel motor 7 918 LUT, 60 DSP). Token başına süre konuma bağlıdır. $t = 0$'da 38 652 çevrimdir; dikkatin her başta $2(t + 1)$ çevrim harcaması nedeniyle test hikâyelerinin en uzun konumunda 79 532 çevrime çıkar. 200 tokenlık üretimin ortalaması 46 612 çevrim/token'dır; bu, 100 MHz'de saniyede yaklaşık 2 145 token demektir.

RTL'in ürettiği hikâye (açgözlü kod çözme; BOS'tan başlayarak 200 token) *Once upon a time, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a big, scary bird in the sky. She was very excited and wanted to see what was inside.*
Lily's mom said, "Let's go to the bird and see what's inside." Lily was so happy and said, "Yes, I will help you."
After a while, Lily's mom told her that it was time to go home, but Lily didn't want to go to the bird. She said, "I'm sorry, Lily. I didn'…

Yukarıdaki metin Verilator'da çalışan RTL'in çıktısıdır. Motor her adımda kendi logitlerinin argmax'ını bir sonraki tokene çevirmiştir. Metin, tamsayı referans modelin açgözlü çıktısıyla token token aynıdır. Float modelin açgözlü çıktısı ise ilk 28 tokendan sonra ayrışır. Açgözlü kod çözme küçük logit farklarını büyüttüğü için bu beklenen bir durumdur; iki metin de dil bilgisi açısından düzgündür.

32 Değerlendirme

- **Bir RTL, on model.** Bigram'dan Llama 2'ye kadar bütün mimariler aynı donanımda, yalnızca bellek içerikleri değiştirilerek çalışır. Bütün modeller bit düzeyinde doğrulanmıştır. Donanımın tek mimari kısıtı baş boyutunun $d_h = P = 8$ olmasıdır.
- **Maliyet nereden geliyor?** Eşit parametre bütçesinde GEMV süresi mimariden bağımsızdır. Farkı normalizasyon, dikkat ve bunların genişlik gerektiren çarpıcıları yapar. Transformer

özelliklerinin donanım maliyeti DSP sayısında açıkça görünür: özelleşmiş motorda yaklaşık 15'e karşılık 50.

- **Kuantizasyonun maliyeti nerede?** Ağırlıklarda değil, aktivasyonlarda. Yüzdelik kalibrasyon ve kalibre NORM ölçeği, donanıma dokunmadan stories260K'daki kaybı yarıdan fazla azaltır.
- **Sınırlamalar.** Hat B'de yalnızca hazır eğitilmiş Llama 2 modeli çalıştırılmıştır; diğer varyantların TinyStories'te eğitilmesi gelecek çalışmadır. Kaynak sayıları zamanlama içermeyen Yosys tahminleridir; Fmax, güç ve kart üzerindeki ölçümler Vivado gerektirir. DSP kullanımı paylaşımlı çarpıcılarla azaltılabilir: NORM ve dikkat birimleri hiçbir zaman aynı anda çalışmaz.

Kaynaklar

[1] A. Vaswani, N. Shazeer, N. Parmar ve ark., "Attention Is All You Need," *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017.

[2] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018.

[3] A. Radford, J. Wu, R. Child ve ark., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.

[4] T. Brown, B. Mann, N. Ryder ve ark., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[5] A. Karpathy, *minGPT*, `https://github.com/karpathy/minGPT`.

[6] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[7] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016.

[8] R. Xiong, Y. Yang, D. He ve ark., "On Layer Normalization in the Transformer Architecture," *ICML*, 2020.

[9] D. Hendrycks, K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016.

[10] I. Loshchilov, F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019.

[11] B. Jacob, S. Kligys, B. Chen ve ark., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018.

[12] M. Nagel, M. Fournarakis, R. A. Amjad ve ark., "A White Paper on Neural Network Quantization," arXiv:2106.08295, 2021.

[13] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, K. Keutzer, "I-BERT: Integer-only BERT Quantization," *ICML*, 2021.

[14] P. A. Samuelson, "How Deviant Can You Be?," *Journal of the American Statistical Association*, 63(324), 1968.

[15] V. Sze, Y.-H. Chen, T.-J. Yang, J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, 105(12), 2017.

[16] R. Pope, S. Douglas, A. Chowdhery ve ark., "Efficiently Scaling Transformer Inference," *MLSys*, 2023.

[17] B. Li, S. Pandey, H. Fang ve ark., "FTRANS: Energy-Efficient Acceleration of Transformers using FPGA," *ISLPED*, 2020.

[18] T. J. Ham, S. J. Jung, S. Kim ve ark., "A^3: Accelerating Attention Mechanisms in Neural Networks with Approximation," *HPCA*, 2020.

[19] Y. Bengio, R. Ducharme, P. Vincent, C. Jauvin, "A Neural Probabilistic Language Model," *Journal of Machine Learning Research*, 3, 2003.

[20] K. Cho, B. van Merriënboer, C. Gulcehre ve ark., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," *EMNLP*, 2014.

[21] B. Zhang, R. Sennrich, "Root Mean Square Layer Normalization," *NeurIPS*, 2019.

[22] J. Su, Y. Lu, S. Pan ve ark., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.

[23] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020.

[24] J. Ainslie, J. Lee-Thorp, M. de Jong ve ark., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *EMNLP*, 2023.

[25] A. Henry, P. R. Dachapally, S. Pawar, Y. Chen, "Query-Key Normalization for Transformers," *Findings of EMNLP*, 2020.

[26] D. R. So, W. Mańke, H. Liu ve ark., "Primer: Searching for Efficient Transformers for Language Modeling," *NeurIPS*, 2021.

[27] H. Touvron, L. Martin, K. Stone ve ark., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv:2307.09288, 2023.

[28] R. Eldan, Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?," arXiv:2305.07759, 2023.

[29] A. Karpathy, *makemore*, `https://github.com/karpathy/makemore`, 2022.

[30] A. Karpathy, *nanoGPT*, `https://github.com/karpathy/nanoGPT`, 2022.

[31] A. Karpathy, *llama2.c*, `https://github.com/karpathy/llama2.c`, 2023.

[32] A. Karpathy, *nanochat*, `https://github.com/karpathy/nanochat`, 2025.

[33] A. Karpathy, "microgpt," `https://karpathy.github.io/2026/02/12/microgpt/`, 2026.